

NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

INFORMATION TECHNOLOGY

ITITE725

PATTERN ANALYSIS AND RECOMMENDER SYSTEMS PRACTICAL FILE

SUBMITTED BY:

2023UIT3007: ARUL JAIN

SUBMITTED TO:

DR. VIKAS MAHESHKAR

INDEX

S.No	Objective
1	Prepare a document by studying any two datasets of different domains, write the description of the dataset consisting of purpose, what is source, write Feature description, how many features, work done by various authors and performance achieved by at least three authors in recent years. Cite References and add references.
2	Prepare a page using flask consisting of a code in python to extract and store the two datasets that you studied in Experiment 1, Plot the performance plot, display different features having description.
3	Prepare an informative page using flask consisting of a code in python to extract features from text/video/audio. Extract at least 3 features from various timestamp (at least 10 Samples) justifying the objective for application. Text/video/audio file should exist in the page.
4	Prepare an interactive page using flask consisting of a code in python for Single feature multiple class, Multiple features multiple class and then classify test samples using Bayesian approach.
5	Prepare a page using flask consisting of a code in python to extract text paragraphs, separate the sentences, remove various stop words, perform Lemmatization and stemming then classify test samples using Bayesian approach.
6	Prepare a page using flask consisting of a code in python to perform Bayesian Risk Minimization for at least three classes, three actions. Given loss function, particular action should trigger particular action, that action should reflect in the page itself.
7	Implement Linear regression in python (no inbuilt function should be used) to find optimized parameters using Gradient descent for any application and Plot Gradient Descent curve.
8	Implement Logistic regression for binary class and multiclass classification in python (no inbuilt function should be used) to find optimized parameters using Gradient descent for any application and Plot Gradient Descent curve.
9	Implement regularization for any application show plots of 1) decision boundary and 2) performance before and after applying regularization.
10	Implement Principal Component Analysis (PCA) for any application which involves testing. Also show the projected line and new reduced features mapped on the projected line.
11	Implement Linear Discriminant Analysis (LDA) for any application which involves testing. Also show the projected line and new reduced features mapped on the projected line.

Experiment 1

Objective

Prepare a document by studying any two datasets of different domains, write the description of the dataset consisting of purpose, what is source, write Feature description, how many features, work done by various authors and performance achieved by at least three authors in recent years. Cite References and add references.

Dataset 1: Breast Cancer Wisconsin (Diagnostic) Dataset

Domain: Healthcare / Medical Diagnosis

Purpose

The dataset is used to build binary classification models that distinguish malignant (cancerous) from benign (non-cancerous) breast tumours using nuclear features extracted from digitized fine needle aspirate (FNA) images. It supports early, accurate computer-aided diagnosis.

Source

Publicly hosted on the UCI Machine Learning Repository (ID 17). Donated by Wolberg, Mangasarian, Street (University of Wisconsin). It contains 569 instances: 357 benign and 212 malignant. [1]

Feature Description

Number of predictor features = 30 (mean, standard error and worst value of 10 nuclear characteristics). Number of target features = 1 (diagnosis). Total = 31 columns.

Feature group	Features
Mean (10)	radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, compactness_mean, concavity_mean, concave_points_mean, symmetry_mean, fractal_dimension_mean
Std. error (10)	radius_se, texture_se, perimeter_se, area_se, smoothness_se, compactness_se, concavity_se, concave_points_se, symmetry_se, fractal_dimension_se
Worst (10)	radius_worst, texture_worst, perimeter_worst, area_worst, smoothness_worst, compactness_worst, concavity_worst,

	concave_points_worst, symmetry_worst, fractal_dimension_worst
Target (1)	diagnosis (M = malignant, B = benign)

Work Done By Various Authors And Performance Achieved

Author(s) / Year	Method	Performance
Street, Wolberg & Mangasarian (1993) [2]	MSM-T decision tree on nuclear features (Xcyt system)	~97% accuracy
Mangasarian, Street & Wolberg (1995) [3]	Diagnosis via linear programming (Operations Research)	~97% (10-fold CV)
Akay (2009) [4]	SVM combined with F-score feature selection	99.51% accuracy
Ensemble study (2024) [5]	AdaBoost-Logistic ensemble of 7 classifiers	99.12% accuracy

Dataset 2: Wine Recognition Dataset

Domain: Food Chemistry / Chemometrics

Purpose

The dataset is used to build multi-class classification models that identify the cultivar of origin (three Italian cultivars) of a wine sample from the chemical analysis of its constituents. It is a benchmark problem with well-separated classes, widely used for first testing of new classifiers.

Source

Publicly hosted on the UCI Machine Learning Repository. Donated by Aeberhard & Forina (1992); chemical analysis performed at the Institute of Pharmaceutical and Food Analysis, University of Genoa, Italy. It contains 178 instances: 59 of cultivar 1, 71 of cultivar 2, 48 of cultivar 3. [6]

Feature Description

Number of predictor features = 13 (all continuous chemical constituents). Number of target features = 1 (cultivar 1/2/3). Total = 14 columns.

No.	Feature	No.	Feature
1	Alcohol	8	Nonflavanoid phenols
2	Malic acid	9	Proanthocyanins
3	Ash	10	Color intensity
4	Alcalinity of ash	11	Hue
5	Magnesium	12	OD280/OD315 of diluted wines
6	Total phenols	13	Proline
7	Flavanoids	Target	Cultivar (1 / 2 / 3)

Work Done By Various Authors And Performance Achieved

Author(s) / Year	Method	Performance
Aeberhard, Coomans & De Vel (1994) [7]	Comparative study of LDA, QDA, k-NN in high dimensions	~98-100% across classifiers
Aeberhard & Forina (1992) [6]	UCI dataset documentation / baseline	Benchmark (well-separated classes)
Reproduced baselines (standardized, 5-fold CV)	Logistic Regression / SVM / k-NN	98.3% / 98.3% / 94.9%

References

[1] Wolberg, W., Mangasarian, O., Street, N. & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic) [Dataset]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

[2] Street, W. N., Wolberg, W. H. & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. Proc. IS&T/SPIE Symposium on Electronic Imaging.

[3] Mangasarian, O. L., Street, W. N. & Wolberg, W. H. (1995). Breast cancer diagnosis and prognosis via linear programming. Operations Research, 43(4), 570-577.

[4] Akay, M. F. (2009). Support vector machines combined with feature selection for breast cancer diagnosis. Expert Systems with Applications, 36(2), 3240-3247.

[5] AdaBoost-Logistic ensemble for breast cancer classification (2024). 99.12% accuracy on WDBC. PMC11005507.

[6] Aeberhard, S. & Forina, M. (1992). Wine [Dataset]. UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/datasets/wine

[7] Aeberhard, S., Coomans, D. & De Vel, O. (1994). Comparative analysis of statistical pattern recognition methods in high dimensional settings. Pattern Recognition, 27(8), 1065-1077.

Experiment 2

Objective

Prepare a page using flask consisting of a code in python to extract and store the two datasets that you studied in Experiment 1, Plot the performance plot, display different features having description.

Theory

The two Experiment 1 datasets are used: Breast Cancer Wisconsin (569 samples, 30 features, healthcare) and Wine Recognition (178 samples, 13 features, food chemistry). A tab toggle switches between them.

Each dataset is extracted with scikit-learn loaders and stored as a CSV file under datasets/ (569x31 and 178x14), satisfying the extract-and-store requirement.

Features are standardized with StandardScaler, then PCA extracts 2 principal components for visualization and classification.

A Logistic Regression classifier is trained on the extracted features and evaluated with accuracy, confusion matrix and one-vs-rest ROC curves (works for 2-class and 3-class targets).

The Flask page shows dataset statistics, PCA scatter plot, confusion matrix, ROC curves and a full feature-description table (mean, std, min, max per feature).

Code

```
from flask import Flask, request, render_template_string
import io
import os
import base64
import numpy as np
import matplotlib
# Use 'Agg' backend so matplotlib doesn't try to open GUI windows in Flask
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine, load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc
```

```
app = Flask(__name__)
BASE = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE, "datasets")
os.makedirs(DATA_DIR, exist_ok=True)

# ============================================
# Datasets studied in Experiment 1
# ============================================
DATASETS = {
    "breast_cancer": {
        "title": "Breast Cancer Wisconsin (Diagnostic)",
        "domain": "Healthcare / Medical Diagnosis",
        "loader": load_breast_cancer,
        "purpose": "Classify tumours as malignant or benign from nuclear features.",
    },
    "wine": {
        "title": "Wine Recognition",
        "domain": "Food Chemistry / Chemometrics",
        "loader": load_wine,
        "purpose": "Identify the cultivar of origin from chemical analysis.",
    },
}

# ============================================
# Helper: Convert Matplotlib Figure to Base64
# ============================================
def fig_to_base64(fig):
    img = io.BytesIO()
    fig.savefig(img, format='png', bbox_inches='tight', dpi=100)
    img.seek(0)
    plt.close(fig)
    return base64.b64encode(img.getvalue()).decode('utf-8')


def load_and_store(key):
    """Extract the dataset and store it as CSV (extract + store)."""
    info = DATASETS[key]
    data = info["loader"]()
    X = np.asarray(data.data, dtype=float)
    y = np.asarray(data.target)
    csv_path = os.path.join(DATA_DIR, f"{key}.csv")
    if not os.path.exists(csv_path):
        header = ",".join(list(data.feature_names) + ["target"])
        np.savetxt(csv_path, np.c_[X, y], delimiter=",", header=header, comments="")
    return data, X, y, csv_path


def multiclass_roc(ax, y_test, y_prob, target_names):
    """One-vs-rest ROC curves (works for binary and 3-class)."""
    n_classes = y_prob.shape[1]
    colors = ["crimson", "darkgreen", "darkorange"]
    for i in range(n_classes):
        binary_true = (y_test == i).astype(int)
```

```
      fpr, tpr, _ = roc_curve(binary_true, y_prob[:, i])
      roc_auc = auc(fpr, tpr)
      ax.plot(fpr, tpr, lw=2, color=colors[i % len(colors)],
              label=f'{target_names[i]} (area = {roc_auc:.2f})')
    ax.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
    ax.set_xlim([0.0, 1.0])
    ax.set_ylim([0.0, 1.05])

# ========================================
# HTML Template (with inline CSS)
# ========================================
HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>Feature Extraction & Performance</title>
    <style>
      body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background-color: #f8e9f0; margin: 0;
padding: 20px; color: #333; }
      .container { max-width: 1200px; margin: auto; }
      .header { text-align: center; margin-bottom: 30px; }
      .header h1 { color: #4a235a; }
      .tabs { text-align: center; margin-bottom: 20px; }
      .tabs a { display: inline-block; padding: 10px 22px; margin: 0 6px; border-radius: 8px; text-decoration: none;
font-weight: bold; color: #4a235a; background: white; border: 2px solid #8e44ad; }
      .tabs a.active { background: #8e44ad; color: white; }
      .stats-panel { display: flex; justify-content: space-between; background: white; padding: 20px; border-radius:
8px; box-shadow: 0 4px 6px rgba(0,0,0,0.05); border-top: 4px solid #8e44ad; margin-bottom: 30px; }
      .stat-box { text-align: center; flex: 1; }
      .stat-box:not(:last-child) { border-right: 1px solid #eee; }
      .stat-value { font-size: 1.8em; font-weight: bold; color: #8e44ad; }
      .stat-label { color: #7f8c8d; font-size: 0.9em; text-transform: uppercase; margin-top: 5px; }

      .charts-container { display: flex; flex-wrap: wrap; gap: 20px; justify-content: space-between; }
      .chart-card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 4px 6px rgba(0,0,0,0.05);
flex: 1; min-width: 350px; text-align: center; }
      .chart-card.full-width { flex: 100%; }
      .chart-card img { max-width: 100%; height: auto; border-radius: 4px; }
      .chart-title { font-weight: bold; font-size: 1.2em; color: #34495e; margin-bottom: 15px; }

      table.feat { width: 100%; border-collapse: collapse; margin-top: 15px; font-size: 0.85em; background: white;
}
      table.feat th, table.feat td { border: 1px solid #ddd; padding: 8px; text-align: left; }
      table.feat th { background: #8e44ad; color: white; }
    </style>
</head>
<body>
    <div class="container">
      <div class="header">
        <h1>Feature Extraction & Performance Analysis</h1>
        <p>Dataset: {{ info.title }} | Domain: {{ info.domain }} | {{ info.purpose }}</p>
      </div>
```

```
    <div class="tabs">
      {% for key, ds in datasets.items() %}
      <a href="/?dataset={{ key }}" class="{{ 'active' if key == active else '' }}">{{ ds.title }}</a>
      {% endfor %}
    </div>

    <div class="stats-panel">
      <div class="stat-box">
        <div class="stat-value">{{ results.original_features }}</div>
        <div class="stat-label">Original Features</div>
      </div>
      <div class="stat-box">
        <div class="stat-value">{{ results.extracted_features }}</div>
        <div class="stat-label">Extracted Features (PCA)</div>
      </div>
      <div class="stat-box">
        <div class="stat-value">{{ results.variance_explained }}</div>
        <div class="stat-label">Data Variance Retained</div>
      </div>
      <div class="stat-box">
        <div class="stat-value" style="color: #27ae60;">{{ results.accuracy }}</div>
        <div class="stat-label">Model Accuracy</div>
      </div>
    </div>

    <div class="charts-container">
      <div class="chart-card full-width">
        <div class="chart-title">1. Feature Extraction: Dimensionality Reduction</div>
        <img src="{{ results.plot_pca }}" alt="PCA Scatter Plot">
      </div>
      <div class="chart-card">
        <div class="chart-title">2. Model Performance: Confusion Matrix</div>
        <img src="{{ results.plot_cm }}" alt="Confusion Matrix">
      </div>
      <div class="chart-card">
        <div class="chart-title">3. Model Performance: ROC Curve</div>
        <img src="{{ results.plot_roc }}" alt="ROC Curve">
      </div>
    </div>

    <div class="chart-card full-width" style="margin-top:20px;">
      <div class="chart-title">4. Feature Description (mean / std / min / max on stored data)</div>
      <table class="feat">
        <tr><th>#</th><th>Feature</th><th>Mean</th><th>Std</th><th>Min</th><th>Max</th></tr>
        {% for f in features %}
        <tr><td>{{ loop.index }}</td><td>{{ f.name }}</td><td>{{ f.mean }}</td>
          <td>{{ f.std }}</td><td>{{ f.min }}</td><td>{{ f.max }}</td></tr>
        {% endfor %}
      </table>
      <p style="color:#7f8c8d;">Target classes: {{ target_names | join(', ') }}</p>
    </div>
  </div>
</body>
```

```
</html>
"""

# ==========================================
# Flask Route & ML Pipeline
# ==========================================
@app.route('/')
def index():
    key = request.args.get('dataset', 'breast_cancer')
    if key not in DATASETS:
        key = 'breast_cancer'
    info = DATASETS[key]

    # 1. Extract + store the dataset
    data, X, y, csv_path = load_and_store(key)
    feature_names = list(data.feature_names)
    target_names = list(data.target_names)
    n_samples, n_features = X.shape

    # 2. Standardization
    X_scaled = StandardScaler().fit_transform(X)

    # 3. Feature Extraction (PCA -> 2 components)
    pca = PCA(n_components=2)
    X_extracted = pca.fit_transform(X_scaled)
    variance_explained = sum(pca.explained_variance_ratio_) * 100

    # 4. Train/Test Split & Model Training
    X_train, X_test, y_train, y_test = train_test_split(
        X_extracted, y, test_size=0.3, random_state=42, stratify=y)
    model = LogisticRegression(max_iter=5000)
    model.fit(X_train, y_train)

    # 5. Performance Evaluation
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)
    accuracy = accuracy_score(y_test, y_pred) * 100

    # Plot 1: PCA Scatter
    fig1, ax1 = plt.subplots(figsize=(7, 5))
    scatter = ax1.scatter(X_extracted[:, 0], X_extracted[:, 1], c=y,
                          cmap='viridis', edgecolors='k', alpha=0.8)
    ax1.set_title("Feature Extraction: PCA (2 Components)")
    ax1.set_xlabel(f"Principal Component 1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
    ax1.set_ylabel(f"Principal Component 2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
    handles, _ = scatter.legend_elements(prop="colors")
    ax1.legend(handles, target_names, title="Class")
    ax1.grid(True, linestyle='--', alpha=0.6)
    plot_pca = fig_to_base64(fig1)

    # Plot 2: Confusion Matrix
    fig2, ax2 = plt.subplots(figsize=(6, 5))
    cm = confusion_matrix(y_test, y_pred)
```

```
    im = ax2.imshow(cm, cmap='Purples')
    fig2.colorbar(im, ax=ax2)
    ax2.set_xticks(range(len(target_names)), target_names)
    ax2.set_yticks(range(len(target_names)), target_names)
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax2.text(j, i, cm[i, j], ha="center", va="center",
                     fontsize=14, fontweight="bold")
    ax2.set_title("Performance: Confusion Matrix")
    ax2.set_xlabel("Predicted Class")
    ax2.set_ylabel("True Class")
    plot_cm = fig_to_base64(fig2)

    # Plot 3: ROC Curve (one-vs-rest, works for 2 or 3 classes)
    fig3, ax3 = plt.subplots(figsize=(6, 5))
    multiclass_roc(ax3, y_test, y_prob, target_names)
    ax3.set_xlabel('False Positive Rate')
    ax3.set_ylabel('True Positive Rate')
    ax3.set_title('Receiver Operating Characteristic (ROC)')
    ax3.legend(loc="lower right")
    ax3.grid(True, linestyle='--', alpha=0.6)
    plot_roc = fig_to_base64(fig3)

    features = [{
        "name": name,
        "mean": f"{X[:, j].mean():.2f}",
        "std": f"{X[:, j].std():.2f}",
        "min": f"{X[:, j].min():.2f}",
        "max": f"{X[:, j].max():.2f}",
    } for j, name in enumerate(feature_names)]

    results = {
        'original_features': n_features,
        'extracted_features': X_extracted.shape[1],
        'variance_explained': f"{variance_explained:.2f}%",
        'accuracy': f"{accuracy:.2f}%",
        'plot_pca': plot_pca,
        'plot_cm': plot_cm,
        'plot_roc': plot_roc,
    }

    return render_template_string(
        HTML_TEMPLATE, info=info, datasets=DATASETS, active=key,
        csv_path=csv_path, n_samples=n_samples, n_features=n_features,
        n_classes=len(target_names), results=results,
        features=features, target_names=target_names)


if __name__ == '__main__':
    print("Starting Data Visualization App (Exp1 datasets)...")
    print("Access the app at: http://127.0.0.1:5000")
    app.run(debug=True)
```

Output

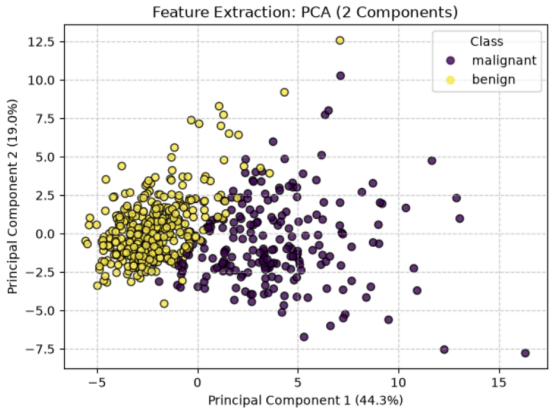
Feature Extraction & Performance Analysis

Dataset: Breast Cancer Wisconsin (Diagnostic) | Domain: Healthcare / Medical Diagnosis | Classify tumours as malignant or benign from nuclear features.

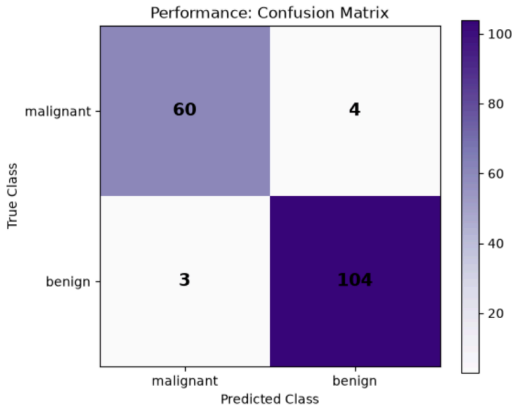
Breast Cancer Wisconsin (Diagnostic) | Wine Recognition

30	2	63.24%	95.91%
ORIGINAL FEATURES	EXTRACTED FEATURES (PCA)	DATA VARIANCE RETAINED	MODEL ACCURACY

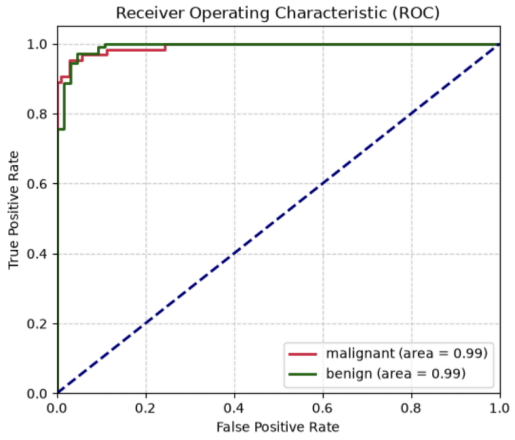
1. Feature Extraction: Dimensionality Reduction



2. Model Performance: Confusion Matrix



3. Model Performance: ROC Curve



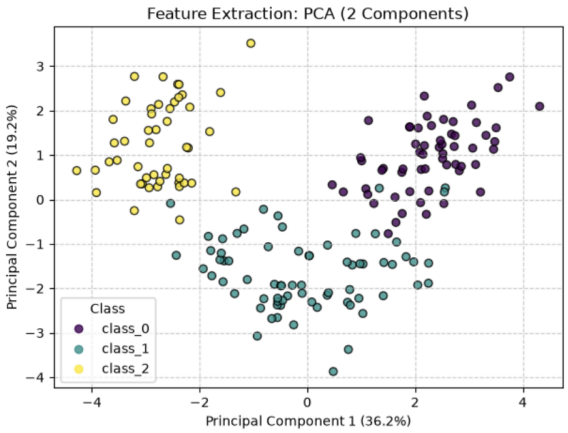
Feature Extraction & Performance Analysis

Dataset: Wine Recognition | Domain: Food Chemistry / Chemometrics | Identify the cultivar of origin from chemical analysis.

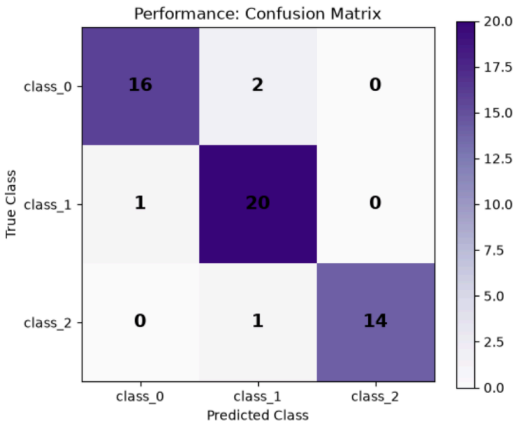
Breast Cancer Wisconsin (Diagnostic) | **Wine Recognition**

13	2	55.41%	92.59%
ORIGINAL FEATURES	EXTRACTED FEATURES (PCA)	DATA VARIANCE RETAINED	MODEL ACCURACY

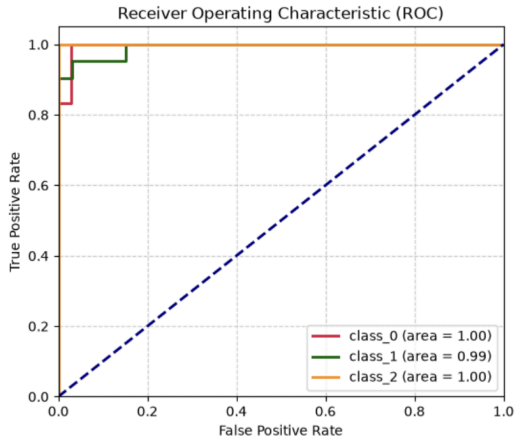
1. Feature Extraction: Dimensionality Reduction



2. Model Performance: Confusion Matrix



3. Model Performance: ROC Curve



Experiment 3

Objective

Prepare an informative page using flask consisting of a code in python to extract features from text/video/audio. Extract at least 3 features from various timestamp (at least 10 Samples) justifying the objective for application. Text/video/audio file should exist in the page.

Theory

One Flask page serves three input types (Audio, Video, Text) with a tab toggle; feature extraction code lives in separate files (audio_features.py, video_features.py, text_features.py) imported by the main app.py.

Audio (.wav/.mp3/.ogg/.flac): each file is split into 1-second windows (at least 10 timestamps) and 3 features are extracted per window - RMS Energy (loudness), Zero Crossing Rate (noisiness) and Spectral Centroid (brightness) - to support podcast speech-activity analysis over time.

Video (.gif via Pillow, .mp4/.avi via OpenCV): frames sampled as timestamps with brightness, contrast, inter-frame motion, edge density and colorfulness per frame, to support lecture-video visual-activity analysis.

Text (.txt): the document is split into sentence/word chunks (at least 10 samples) with word count, average word length and lexicon-based sentiment polarity per chunk, to support sentiment-trend analysis over document position.

Every uploaded file is embedded back in the page (audio player, video player/image, text preview) alongside its numeric feature table.

Code

exp3/app.py

```
"""Exp3 MAIN app: one Flask page with toggle for Audio / Video / Text.

Feature code lives in separate files:
  audio_features.py, video_features.py, text_features.py
Run:  python app.py   ->  http://127.0.0.1:5000
"""
import os

from flask import Flask, render_template_string, request, send_from_directory, url_for
```

```
from werkzeug.utils import secure_filename

from audio_features import extract_audio_features
from text_features import extract_text_features
from video_features import extract_video_features

BASE = os.path.dirname(os.path.abspath(__file__))
UPLOAD_DIR = os.path.join(BASE, "uploads")
os.makedirs(UPLOAD_DIR, exist_ok=True)

app = Flask(__name__)

HTML = """
<!DOCTYPE html><html><head><title>Exp 3: Feature Extraction (Audio/Video/Text)</title>
<style>
body{font-family:'Segoe UI',Tahoma,sans-serif;background:#f4f7f6;margin:0;padding:28px;color:#333}
.container{max-width:1080px;margin:auto}
.card{background:#fff;padding:24px;border-radius:10px;box-shadow:0 4px 10px
rgba(0,0,0,.06);margin-bottom:20px;border-top:5px solid #2ecc71}
h1,h2{color:#2c3e50;margin-top:0}
.tabs{display:flex;gap:10px;margin:14px 0}
.tabbtn{padding:10px 22px;border:2px solid
#2c3e50;background:#fff;color:#2c3e50;border-radius:8px;cursor:pointer;font-weight:bold}
.tabbtn.active{background:#2c3e50;color:#fff}
.panel{display:none}.panel.active{display:block}
table{width:100%;border-collapse:collapse;margin-top:14px;font-size:.88em}
th,td{border:1px solid #eee;padding:9px;text-align:left;vertical-align:top}
th{background:#2c3e50;color:#fff}
tr:nth-child(even){background:#f9f9f9}
button.go{background:#2ecc71;color:#fff;border:none;padding:10px
18px;cursor:pointer;border-radius:5px;font-weight:bold}
audio,video,img.preview{width:100%;margin-top:10px;border-radius:8px}
pre{background:#f7f9f9;border:1px solid
#e5e8e8;padding:12px;border-radius:8px;max-height:220px;overflow:auto;white-space:pre-wrap}
code{background:#eef;padding:2px 6px;border-radius:4px}
</style>
<script>
function
tab(name){document.querySelectorAll('.tabbtn').forEach(b=>b.classList.toggle('active',b.dataset.t===name));
document.querySelectorAll('.panel').forEach(p=>p.classList.toggle('active',p.id==='p-'+name));}
window.addEventListener('DOMContentLoaded',()=>tab('{{ atab }}'));
</script></head><body><div class="container">
<div class="card">
<h1>Experiment 3: Feature Extraction over Timestamps</h1>
<p>One page, three input types. Pick a tab, upload a file (it stays embedded in the page),
get <b>&ge;3 features &times; &ge;10 timestamp samples</b> with an application justification.</p>
<div class="tabs">
<button class="tabbtn" data-t="audio" onclick="tab('audio')">Audio</button>
<button class="tabbtn" data-t="video" onclick="tab('video')">Video</button>
<button class="tabbtn" data-t="text" onclick="tab('text')">Text</button>
</div>
<div class="panel" id="p-audio">
<p><b>Objective:</b> podcast speech-activity analysis over time. Features:
```

```
<code>RMS</code> (loudness), <code>ZCR</code> (noisiness), <code>Spectral Centroid</code>
(brightness).</p>
<form method="POST" enctype="multipart/form-data">
<input type="hidden" name="input_type" value="audio">
<input type="file" name="file" accept=".wav,.mp3,.ogg,.flac,audio/*" required>
<button class="go" type="submit">Extract</button></form></div>
<div class="panel" id="p-video">
<p><b>Objective:</b> lecture-video visual-activity analysis over frames. Features per frame:
<code>Brightness</code>, <code>Contrast</code>, <code>Motion vs prev frame</code> (+ edge,
colorfulness).</p>
<form method="POST" enctype="multipart/form-data">
<input type="hidden" name="input_type" value="video">
<input type="file" name="file" accept=".gif,.mp4,.avi,.mov,.mkv,image/gif,video/*" required>
<button class="go" type="submit">Extract</button></form></div>
<div class="panel" id="p-text">
<p><b>Objective:</b> review/transcript sentiment trend over document position. Features per chunk:
<code>Word count</code>, <code>Avg word length</code>, <code>Sentiment polarity</code>.</p>
<form method="POST" enctype="multipart/form-data">
<input type="hidden" name="input_type" value="text">
<input type="file" name="file" accept=".txt,text/plain" required>
<button class="go" type="submit">Extract</button></form></div>
</div>
{% if error %}<div class="card" style="border-top-color:#e74c3c"><b style="color:#e74c3c">{{ error
}}</b></div>{% endif %}
{% if audio %}
<div class="card" style="border-top-color:#3498db">
<h2>Audio file (embedded) + {{ audio.rows|length }} timestamp samples</h2>
<p><b>{{ afname }}</b> | {{ audio.sr }} Hz | {{ audio.duration }} s | window {{ audio.window }} s</p>
<audio controls src="{{ audiourl }}"></audio>
<table><tr><th>#</th><th>Timestamp</th><th>RMS</th><th>ZCR</th><th>Centroid (Hz)</th></tr>
{% for r in audio.rows %}<tr><td>{{ loop.index }}</td><td>{{ r.timestamp }}</td><td>{{ r.rms }}</td>
<td>{{ r.zcr }}</td><td>{{ r.centroid }}</td></tr>{% endfor %}</table>
</div>{% endif %}
{% if video %}
<div class="card" style="border-top-color:#9b59b6">
<h2>Video file (embedded) + {{ video.rows|length }} frame samples</h2>
<p><b>{{ vfname }}</b> | {{ video.source }} | total frames {{ video.n_frames_total }} | ~{{ video.duration }}
s</p>
{% if vfname.endswith('.gif') %}<img class="preview" src="{{ videourl }}">{% else %}<video class="preview"
controls preload="metadata"><source src="{{ videourl }}" type="video/mp4"></video>{% endif %}
<table><tr><th>#</th><th>Frame /
Time</th><th>Brightness</th><th>Contrast</th><th>Motion</th><th>Edge</th><th>Color</th></tr>
{% for r in video.rows %}<tr><td>{{ loop.index }}</td><td>{{ r.frame }} / {{ r.timestamp }}</td><td>{{
r.brightness }}</td>
<td>{{ r.contrast }}</td><td>{{ r.motion }}</td><td>{{ r.edge }}</td><td>{{ r.colorfulness }}</td></tr>{%
endfor %}</table>
</div>{% endif %}
{% if text %}
<div class="card" style="border-top-color:#e67e22">
<h2>Text file (embedded) + {{ text.rows|length }} samples ({{ text.mode }})</h2>
<p><b>{{ tfname }}</b> | {{ text.n_chars }} chars</p>
<pre>{{ text.full_text[:2000] }}</pre>
<table><tr><th>#</th><th>Sample</th><th>Words</th><th>Avg
```

```
len</th><th>Polarity</th><th>Preview</th></tr>
{% for r in text.rows %}<tr><td>{{ loop.index }}</td><td>{{ r.sample }}</td><td>{{ r.words }}</td>
<td>{{ r.avg_len }}</td><td>{{ r.polarity }}</td><td>{{ r.preview }}</td></tr>{% endfor %}</table>
</div>{% endif %}
</div></body></html>
"""


@app.route("/uploads/<path:f>")
def up(f):
  return send_from_directory(UPLOAD_DIR, f)


@app.route("/", methods=["GET", "POST"])
def index():
  atab, error = "audio", None
  audio = video = text = None
  afname = vfname = tfname = audiourl = videourl = None
  if request.method == "POST":
    kind = request.form.get("input_type", "audio")
    atab = kind if kind in ("audio", "video", "text") else "audio"
    try:
      if kind == "audio":
        f = request.files.get("file")
        if not f or not f.filename:
          raise ValueError("No audio file selected.")
        afname = secure_filename(f.filename)
        f.save(os.path.join(UPLOAD_DIR, afname))
        audio = extract_audio_features(os.path.join(UPLOAD_DIR, afname))
        audiourl = url_for("up", f=afname)
        if len(audio["rows"]) < 10:
          error = f"Only {len(audio['rows'])} samples - upload a longer WAV for >=10."
      elif kind == "video":
        f = request.files.get("file")
        if not f or not f.filename:
          raise ValueError("No video file selected.")
        vfname = secure_filename(f.filename)
        f.save(os.path.join(UPLOAD_DIR, vfname))
        video = extract_video_features(os.path.join(UPLOAD_DIR, vfname))
        videourl = url_for("up", f=vfname)
        if len(video["rows"]) < 10:
          error = f"Only {len(video['rows'])} frames - upload a longer clip for >=10."
      else:
        f = request.files.get("file")
        if not f or not f.filename:
          raise ValueError("No text file selected.")
        tfname = secure_filename(f.filename)
        f.save(os.path.join(UPLOAD_DIR, tfname))
        text = extract_text_features(os.path.join(UPLOAD_DIR, tfname))
        if len(text["rows"]) < 10:
          error = f"Only {len(text['rows'])} samples - upload a longer document for >=10."
    except Exception as e:
      error = f"Failed: {e}"
```

```
    return render_template_string(HTML, atab=atab, error=error,
                                  audio=audio, afname=afname, audiourl=audiourl,
                                  video=video, vfname=vfname, videourl=videourl,
                                  text=text, tfname=tfname)


if __name__ == "__main__":
    print("Open http://127.0.0.1:5000")
    app.run(debug=True)
```

exp3/audio_features.py

```
"""Exp3 - audio feature extraction over timestamps (numpy + soundfile for MP3)."""
import os
import struct
import wave

import numpy as np


def _load_wav_wave(filepath):
    """Fast stdlib WAV loader. Returns (mono float32 data, samplerate)."""
    with wave.open(filepath, "rb") as w:
        n_channels = w.getnchannels()
        sampwidth = w.getsampwidth()
        framerate = w.getframerate()
        n_frames = w.getnframes()
        raw = w.readframes(n_frames)
    if sampwidth == 1:  # 8-bit unsigned
        data = np.array(struct.unpack(f"{n_frames * n_channels}B", raw),
                        dtype=np.float32)
        data = (data - 128.0) / 128.0
    elif sampwidth == 2:  # 16-bit signed
        data = np.array(struct.unpack(f"<{n_frames * n_channels}h", raw),
                        dtype=np.float32) / 32768.0
    else:
        raise ValueError(f"Unsupported sample width {sampwidth} (use 8/16-bit WAV)")
    if n_channels > 1:
        data = data.reshape(-1, n_channels).mean(axis=1)
    return data, framerate


def _load_audio(filepath):
    """Load WAV via stdlib; MP3/OGG/FLAC via soundfile. Returns (mono, sr)."""
    ext = os.path.splitext(filepath)[1].lower()
    if ext == ".wav":
        try:
            return _load_wav_wave(filepath)
        except Exception:
            pass  # fall through to soundfile for odd WAV encodings
    try:
```

```
        import soundfile as sf
    except ImportError:
        raise ValueError(f"{ext or 'audio'} needs `pip install soundfile` "
                         "(WAV works without it).")
    data, sr = sf.read(filepath, dtype="float32", always_2d=False)
    data = np.asarray(data, dtype=np.float32)
    if data.ndim > 1:
        data = data.mean(axis=1)
    return data, sr


def extract_audio_features(filepath, window_sec=1.0, min_samples=10):
    """Split audio into windows and extract 3 features per timestamp.

    Supports WAV (stdlib) + MP3/OGG/FLAC (via soundfile).
    Returns dict with sr/duration/window/rows.
    Row: {timestamp, t_start, rms, zcr, centroid}
    """
    data, framerate = _load_audio(filepath)
    duration = len(data) / float(framerate)

    if duration / window_sec < min_samples:  # guarantee >=10 timestamps
        window_sec = duration / min_samples
    win = max(1, int(framerate * window_sec))
    n_win = int(len(data) // win)

    rows = []
    for i in range(n_win):
        seg = data[i * win:(i + 1) * win]
        t_start = round(i * window_sec, 2)
        t_end = round((i + 1) * window_sec, 2)
        rms = float(np.sqrt(np.mean(seg ** 2)))                    # loudness
        zcr = float(np.mean(np.abs(np.diff(np.sign(seg)))) / 2.0)    # noisiness
        windowed = seg * np.hanning(len(seg))                       # brightness
        spec = np.abs(np.fft.rfft(windowed))
        freqs = np.fft.rfftfreq(len(seg), 1.0 / framerate)
        centroid = float(np.sum(freqs * spec) / (np.sum(spec) + 1e-10))
        rows.append({"timestamp": f"{t_start:.2f} - {t_end:.2f} s",
                     "t_start": t_start, "rms": round(rms, 4),
                     "zcr": round(zcr, 4), "centroid": round(centroid, 1)})
    return {"sr": framerate, "channels": 1 if data.ndim == 1 else data.shape[1],
            "duration": round(duration, 2), "window": round(window_sec, 2),
            "rows": rows}
```

exp3/video_features.py

```
"""Exp3 - video feature extraction over frames (Pillow + numpy; cv2 optional).

GIF path works with current lab deps (no opencv needed).
MP4/AVI path uses cv2 only if installed, else raises a clear message.
Features per frame timestamp: brightness, contrast, motion vs prev frame.
```

```
Application: lecture-video scene analysis (static slide vs demo vs cut).
"""
import os

import numpy as np
from PIL import Image


def _frame_feats(gray, rgb, prev_gray):
    gray = gray.astype(np.float32)
    brightness = float(gray.mean())                    # mean intensity
    contrast = float(gray.std())                       # intensity spread
    gx = np.abs(np.diff(gray, axis=1)).mean()          # edge density
    gy = np.abs(np.diff(gray, axis=0)).mean()
    edge = round(float((gx + gy) / 2), 2)
    if prev_gray is None:
        motion = 0.0
    else:
        motion = float(np.abs(gray - prev_gray).mean())   # inter-frame change
    r, g, b = rgb[:, :, 0].astype(np.float32), rgb[:, :, 1].astype(np.float32), rgb[:, :, 2].astype(np.float32)
    colorfulness = round(float(np.abs(r - g).mean() + np.abs(g - b).mean()), 1)
    return round(brightness, 1), round(contrast, 1), round(motion, 2), edge, colorfulness


def _extract_gif(filepath, max_frames=12):
    im = Image.open(filepath)
    frame_ms = im.info.get("duration", 200) or 200
    n = getattr(im, "n_frames", 1)
    if n <= max_frames:
        idx = list(range(n))  # take every frame; short GIFs keep all frames
    else:
        idx = sorted(set(np.linspace(0, n - 1, max(max_frames, 10)).astype(int).tolist()))
    rows, prev = [], None
    for k, fi in enumerate(idx):
        im.seek(fi)
        fr = im.convert("RGB").resize((160, 120))
        gray = np.array(fr.convert("L"))
        rgb = np.array(fr)
        b, c, m, e, col = _frame_feats(gray, rgb, prev)
        ts = fi * frame_ms / 1000.0
        rows.append({"frame": fi, "timestamp": f"{ts:.2f} s",
                     "t_start": round(ts, 2), "brightness": b,
                     "contrast": c, "motion": m, "edge": e,
                     "colorfulness": col})
        prev = gray.astype(np.float32)
    dur = n * frame_ms / 1000.0
    return {"source": "GIF/Pillow", "n_frames_total": n,
            "duration": round(dur, 2), "rows": rows}


def _extract_cv2(filepath, max_frames=12):
    import cv2  # optional
    cap = cv2.VideoCapture(filepath)
```

```
    if not cap.isOpened():
        raise ValueError("Could not open video (unsupported codec?).")
    fps = cap.get(cv2.CAP_PROP_FPS) or 10.0
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    take = sorted(set(np.linspace(0, max(0, total - 1),
                                  min(total or max_frames, max(10, max_frames))).astype(int).tolist()))
    if not take:
        raise ValueError("No frames found.")
    rows, prev, k = [], None, 0
    for fi in take:
        cap.set(cv2.CAP_PROP_POS_FRAMES, fi)
        ok, frame = cap.read()
        if not ok:
            continue
        frame = cv2.resize(frame, (160, 120))
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        b, c, m, e, col = _frame_feats(gray, rgb, prev)
        rows.append({"frame": fi, "timestamp": f"{fi / fps:.2f} s",
                     "t_start": round(fi / fps, 2), "brightness": b,
                     "contrast": c, "motion": m, "edge": e,
                     "colorfulness": col})
        prev = gray.astype(np.float32)
        k += 1
    cap.release()
    return {"source": "Video/cv2", "n_frames_total": total,
            "duration": round(total / fps, 2) if total else 0.0, "rows": rows}


def extract_video_features(filepath, max_frames=12):
    ext = os.path.splitext(filepath)[1].lower()
    if ext in (".gif", ".png", ".apng", ".webp", ".bmp", ".jpg", ".jpeg"):
        return _extract_gif(filepath, max_frames)
    try:
        return _extract_cv2(filepath, max_frames)
    except ImportError:
        raise ValueError("MP4/AVI needs opencv (pip install opencv-python-headless). "
                         "Otherwise upload a .gif.")
```

exp3/text_features.py

```
"""Exp3 - text feature extraction over chunks (stdlib only).

Treats each sentence/chunk as a timestamp sample so we get >=10 samples.
Features per sample: word count, avg word length, sentiment polarity.
Application: review/transcript sentiment trend over document position.
"""
import re

POS = {"good", "great", "excellent", "love", "loved", "amazing", "awesome",
       "best", "nice", "happy", "like", "liked", "wonderful", "perfect",
```

```
    "brilliant", "fantastic", "enjoy", "enjoyed", "recommend", "superb"}
NEG = {"bad", "poor", "terrible", "awful", "hate", "hated", "worst", "boring",
    "disappoint", "disappointed", "waste", "horrible", "annoying", "slow",
    "weak", "lame", "flop", "dull", "pathetic", "useless"}


def _polarity(words):
  if not words:
    return 0.0
  p = sum(1 for w in words if w in POS)
  n = sum(1 for w in words if w in NEG)
  return round((p - n) / len(words), 3)


def extract_text_features(filepath, min_samples=10):
  with open(filepath, "r", encoding="utf-8", errors="replace") as f:
    text = f.read()
  text = text.strip()
  if not text:
    raise ValueError("Empty text file.")

  sentences = [s.strip() for s in
          re.split(r"(?<=[.!?])\s+", text) if s.strip()]
  if len(sentences) >= min_samples:
    chunks = sentences
    mode = "sentence"
  else:  # fall back to equal word-chunks so we always reach >=10 samples
    words_all = text.split()
    k = min_samples
    size = max(1, (len(words_all) + k - 1) // k)
    chunks = [" ".join(words_all[i:i + size])
          for i in range(0, len(words_all), size)]
    mode = "word-chunk"

  rows = []
  for i, ch in enumerate(chunks):
    words = re.findall(r"[A-Za-z']+", ch.lower())
    wc = len(words)
    avg_len = round(sum(len(w) for w in words) / wc, 2) if wc else 0.0
    pol = _polarity(words)
    rows.append({"sample": f"{i + 1:02d} ({mode} {i + 1}/{len(chunks)})",
            "t_start": i, "words": wc, "avg_len": avg_len,
            "polarity": pol,
            "preview": ch[:90] + ("..." if len(ch) > 90 else "")})
  return {"n_chars": len(text), "n_samples": len(rows),
      "mode": mode, "rows": rows, "full_text": text}
```

Output

Experiment 3: Feature Extraction over Timestamps

One page, three input types. Pick a tab, upload a file (it stays embedded in the page), get **≥3 features × ≥10 timestamp samples** with an application justification.

Audio | Video | Text

Objective: podcast speech-activity analysis over time. Features: `RMS` (loudness), `ZCR` (noisiness), `Spectral Centroid` (brightness).

Choose File | No file chosen Extract

Audio file (embedded) + 15 timestamp samples

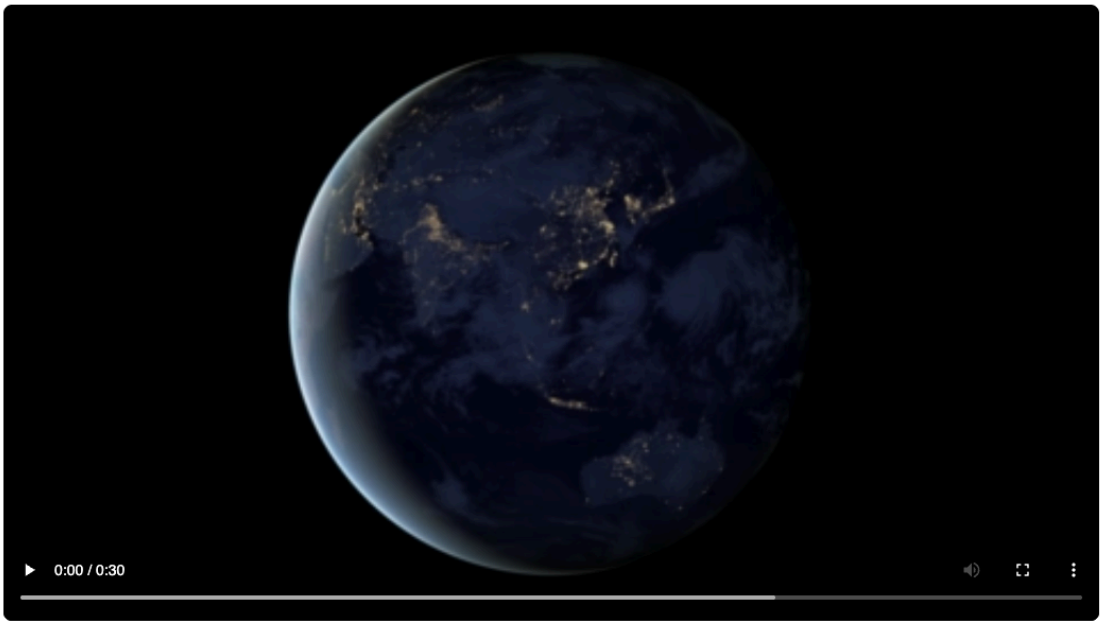
freesound_community-random-beep-76811.mp3 | 24000 Hz | 15.65 s | window 1.0 s

0:00 / 0:15

#	Timestamp	RMS	ZCR	Centroid (Hz)
1	0.00 - 1.00 s	0.2866	0.212	3248.5
2	1.00 - 2.00 s	0.2674	0.2445	3409.6
3	2.00 - 3.00 s	0.2768	0.1962	3134.5
4	3.00 - 4.00 s	0.2791	0.2407	3389.5
5	4.00 - 5.00 s	0.2899	0.1556	2779.9
6	5.00 - 6.00 s	0.256	0.2622	3162.2
7	6.00 - 7.00 s	0.2846	0.2236	2939.0
8	7.00 - 8.00 s	0.2862	0.1838	2841.2
9	8.00 - 9.00 s	0.3057	0.1953	3125.7
10	9.00 - 10.00 s	0.309	0.1828	2742.4
11	10.00 - 11.00 s	0.3211	0.1473	2469.2
12	11.00 - 12.00 s	0.3117	0.211	2969.6
13	12.00 - 13.00 s	0.2798	0.2316	3386.4
14	13.00 - 14.00 s	0.2987	0.2078	3099.2
15	14.00 - 15.00 s	0.2945	0.2026	2325.8

Video file (embedded) + 12 frame samples

file_example_MP4_480_1_5MG.mp4 | Video/cv2 | total frames 901 | ~30.03 s



#	Frame / Time	Brightness	Contrast	Motion	Edge	Color
1	0 / 0.00 s	8.5	21.0	0.0	2.67	7.4
2	81 / 2.70 s	9.2	21.5	3.29	2.77	7.6
3	163 / 5.43 s	9.4	21.0	3.51	2.82	7.9
4	245 / 8.17 s	8.9	21.3	3.72	2.72	7.6
5	327 / 10.90 s	8.6	20.7	3.84	2.59	7.5
6	409 / 13.63 s	8.2	19.9	3.79	2.64	7.5
7	490 / 16.33 s	7.7	19.6	3.59	2.56	7.3
8	572 / 19.07 s	7.1	19.5	2.97	2.39	7.1
9	654 / 21.80 s	6.7	19.3	2.42	2.2	7.0
10	736 / 24.53 s	6.9	19.3	2.22	2.18	7.1
11	818 / 27.27 s	7.7	20.0	2.53	2.42	7.3
12	900 / 30.00 s	8.5	21.0	3.01	2.47	7.3

Experiment 4

Objective

Prepare an interactive page using flask consisting of a code in python for Single feature multiple class, Multiple features multiple class and then classify test samples using Bayesian approach.

Theory

A Discrete (Categorical) Naive Bayes classifier is built from scratch for the application of predicting the mode of transport (Bike, Car, Bus).

Part 1 uses a single discrete feature (Weather) to classify among the three classes using P(C|x) proportional to P(C)P(x|C).

Part 2 uses multiple discrete features (Weather, Distance, Traffic) with the Naive Bayes independence assumption: P(C|X) proportional to P(C) times the product of P(xi|C).

Laplace smoothing (adding 1 to all counts) ensures unseen feature-class combinations get a small non-zero probability instead of zero.

Posteriors are normalized to percentages and the class with the highest posterior is selected; the Flask page takes test samples through dropdown forms and displays the full posterior table.

Code

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

# ========================================
# 1. Discrete Naive Bayes Engine (From Scratch)
# ========================================
class DiscreteNaiveBayes:
    def __init__(self):
        self.classes = []
        self.priors = {}
        self.likelihoods = {}

    def fit(self, X, y):
        self.classes = list(set(y))
        total_samples = len(y)
```

```
        # Calculate Prior Probabilities P(Class)
        self.priors = {c: y.count(c) / total_samples for c in self.classes}

        num_features = len(X[0])

        # Calculate Likelihoods P(Feature | Class) with Laplace Smoothing
        for feature_idx in range(num_features):
            self.likelihoods[feature_idx] = {}
            unique_values = set(row[feature_idx] for row in X)
            num_unique = len(unique_values)

            for c in self.classes:
                self.likelihoods[feature_idx][c] = {}

                # Get all feature values where the target class matches 'c'
                feature_vals_for_class = [X[i][feature_idx] for i in range(total_samples) if y[i] == c]
                class_count = len(feature_vals_for_class)

                for val in unique_values:
                    val_count = feature_vals_for_class.count(val)
                    # Laplace Smoothing: (count + 1) / (class_count + num_unique)
                    smoothed_prob = (val_count + 1) / (class_count + num_unique)
                    self.likelihoods[feature_idx][c][val] = smoothed_prob

    def predict_proba(self, x_input):
        posteriors = {}

        for c in self.classes:
            # Start with the prior probability P(C)
            prob = self.priors[c]

            # Multiply by the likelihood of each discrete feature P(x_i | C)
            for feature_idx, val in enumerate(x_input):
                # If a value was never seen in training, assign a tiny probability
                likelihood = self.likelihoods[feature_idx][c].get(val, 1e-6)
                prob *= likelihood

            posteriors[c] = prob

        # Normalize to percentages
        total_prob = sum(posteriors.values())
        return {c: (p / total_prob) * 100 for c, p in posteriors.items()}

# ==========================================
# 2. Datasets & Model Training
# Application: Predicting Transportation Mode
# ==========================================

# PART 1: Single Discrete Feature -> 3 Classes
# Feature: Weather | Target: Mode of Transport
X_single = [['Sunny'], ['Sunny'], ['Rain'], ['Rain'], ['Overcast'], ['Overcast'], ['Sunny'], ['Rain'], ['Overcast']]
y_single = ['Bike', 'Car', 'Bus', 'Car', 'Bike', 'Bus', 'Bike', 'Bus', 'Car']
```

```
model_part1 = DiscreteNaiveBayes()
model_part1.fit(X_single, y_single)


# PART 2: Multiple Discrete Features -> 3 Classes
# Features: [Weather, Distance, Traffic] | Target: Mode of Transport
X_multi = [
   ['Sunny', 'Short', 'Low'],
   ['Sunny', 'Long', 'High'],
   ['Rain', 'Short', 'High'],
   ['Rain', 'Long', 'Low'],
   ['Overcast', 'Short', 'Low'],
   ['Overcast', 'Long', 'High'],
   ['Sunny', 'Medium', 'Low'],
   ['Rain', 'Medium', 'High'],
   ['Overcast', 'Medium', 'Low']
]
y_multi = ['Bike', 'Car', 'Bus', 'Bus', 'Bike', 'Car', 'Bike', 'Bus', 'Car']

model_part2 = DiscreteNaiveBayes()
model_part2.fit(X_multi, y_multi)

# ==========================================
# 3. Flask HTML Template & Routing
# ==========================================
HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
   <title>Discrete Bayesian Classification</title>
   <style>
      body { font-family: Arial, sans-serif; background: #f4f7f6; margin: 0; padding: 30px; }
      .container { max-width: 900px; margin: auto; }
      .card { background: white; padding: 25px; border-radius: 10px; box-shadow: 0 5px 15px rgba(0,0,0,0.08);
margin-bottom: 30px; border-left: 5px solid #8e44ad; }
      h1, h2 { color: #2c3e50; }
      .result-box { background: #fdfefe; padding: 15px; border-radius: 8px; margin-top: 15px; border: 1px solid
#dcdcdc; }
      .winner { font-size: 1.3em; font-weight: bold; color: #27ae60; }
      select { padding: 8px; width: 150px; border: 1px solid #ccc; border-radius: 4px; margin-right: 15px; }
      button { background: #8e44ad; color: white; border: none; padding: 10px 20px; cursor: pointer;
border-radius: 5px; font-weight: bold; }
      button:hover { background: #9b59b6; }
      table { width: 100%; border-collapse: collapse; margin-top: 15px; }
      th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
      th { background: #f9f9f9; }
   </style>
</head>
<body>
   <div class="container">
      <h1>Experiment 4: Discrete Bayesian Classification</h1>

      <!-- PART 1: Single Feature -->
```

```
    <div class="card">
      <h2>Part 1: Single Value, Multiple Class</h2>
      <p><strong>Scenario:</strong> Predict transportation mode based ONLY on Weather.</p>
      <form method="POST">
        <input type="hidden" name="form_type" value="part1">
        <label>Weather: </label>
        <select name="weather1">
          <option value="Sunny">Sunny</option>
          <option value="Overcast">Overcast</option>
          <option value="Rain">Rain</option>
        </select>
        <button type="submit">Classify</button>
      </form>

      {% if result1 %}
      <div class="result-box">
        <h3>Classification Results</h3>
        <table>
          <tr><th>Class (Mode)</th><th>Posterior Probability P(C|X)</th></tr>
          {% for cls, prob in result1.items() %}
          <tr>
            <td>{{ cls }}</td>
            <td>{{ "%.2f"|format(prob) }}%</td>
          </tr>
          {% endfor %}
        </table>
        <p>Selected Mode: <span class="winner">{{ winner1 }}</span></p>
      </div>
      {% endif %}
    </div>

    <!-- PART 2: Multiple Features -->
    <div class="card" style="border-left-color: #2980b9;">
      <h2>Part 2: Multiple Value, Multiple Class</h2>
      <p><strong>Scenario:</strong> Predict transportation mode based on Weather, Distance, and
Traffic.</p>
      <form method="POST">
        <input type="hidden" name="form_type" value="part2">

        <label>Weather: </label>
        <select name="weather2">
          <option value="Sunny">Sunny</option>
          <option value="Overcast">Overcast</option>
          <option value="Rain">Rain</option>
        </select>

        <label>Distance: </label>
        <select name="distance">
          <option value="Short">Short</option>
          <option value="Medium">Medium</option>
          <option value="Long">Long</option>
        </select>
```

```
            <label>Traffic: </label>
            <select name="traffic">
              <option value="Low">Low</option>
              <option value="High">High</option>
            </select>

            <button type="submit" style="background: #2980b9;">Classify</button>
          </form>

          {% if result2 %}
          <div class="result-box">
            <h3>Classification Results</h3>
            <table>
              <tr><th>Class (Mode)</th><th>Posterior Probability P(C|X)</th></tr>
              {% for cls, prob in result2.items() %}
              <tr>
                <td>{{ cls }}</td>
                <td>{{ "%.2f"|format(prob) }}%</td>
              </tr>
              {% endfor %}
            </table>
            <p>Selected Mode: <span class="winner">{{ winner2 }}</span></p>
          </div>
          {% endif %}
        </div>
      </div>
    </body>
    </html>
    """

    @app.route('/', methods=['GET', 'POST'])
    def index():
      result1, winner1 = None, None
      result2, winner2 = None, None

      if request.method == 'POST':
        form_type = request.form.get('form_type')

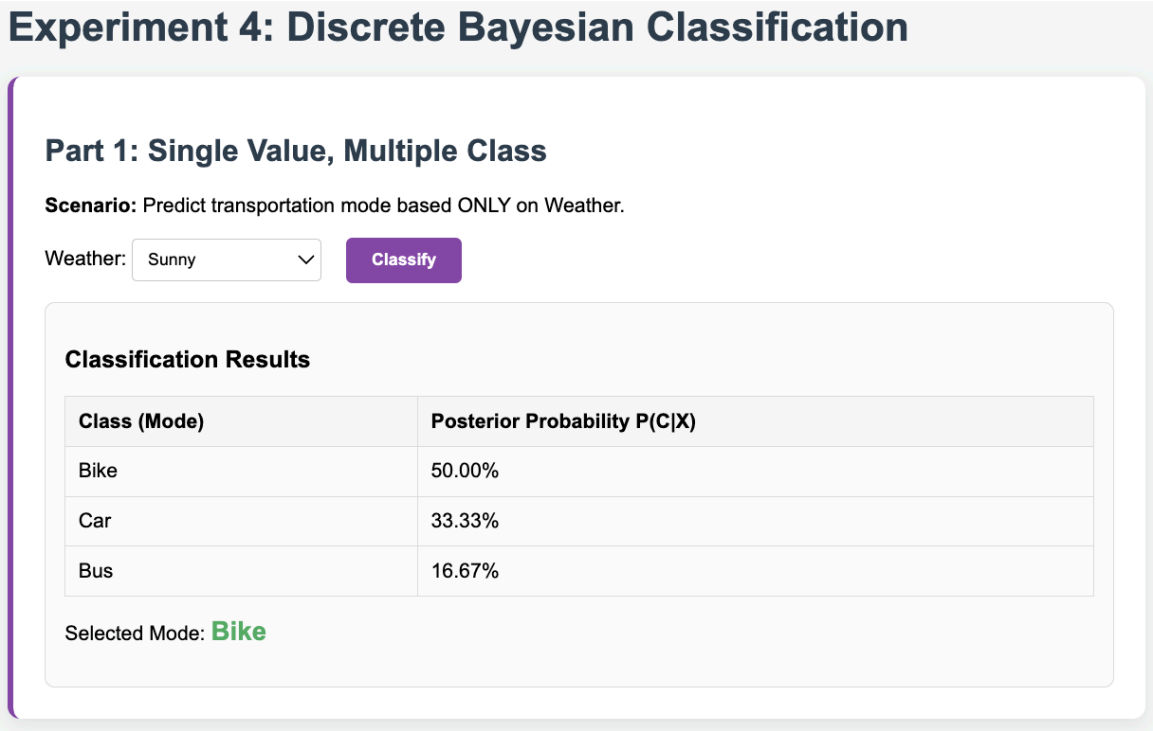
        if form_type == 'part1':
          x_input = [request.form.get('weather1')]
          probs = model_part1.predict_proba(x_input)
          result1 = dict(sorted(probs.items(), key=lambda item: item[1], reverse=True))
          winner1 = max(probs, key=probs.get)

        elif form_type == 'part2':
          x_input = [
            request.form.get('weather2'),
            request.form.get('distance'),
            request.form.get('traffic')
          ]
          probs = model_part2.predict_proba(x_input)
          result2 = dict(sorted(probs.items(), key=lambda item: item[1], reverse=True))
          winner2 = max(probs, key=probs.get)
```

```
    return render_template_string(HTML_TEMPLATE,
                                  result1=result1, winner1=winner1,
                                  result2=result2, winner2=winner2)

if __name__ == '__main__':
    print("Starting Discrete Bayesian Classifier App...")
    print("Access at: http://127.0.0.1:5000")
    app.run(debug=True)
```

Output



Part 2: Multiple Value, Multiple Class

Scenario: Predict transportation mode based on Weather, Distance, and Traffic.

Weather: Sunny Distance: Short Traffic: Low Classify

Classification Results

Class (Mode)	Posterior Probability P(C|X)
Bike	81.82%
Car	9.09%
Bus	9.09%

Selected Mode: **Bike**

Experiment 5

Objective

Prepare a page using flask consisting of a code in python to extract text paragraphs, separate the sentences, remove various stop words, perform Lemmatization and stemming then classify test samples using Bayesian approach.

Theory

The input paragraph is split into individual sentences with sent_tokenize, so each sentence becomes a test sample for classification.

Each sentence is lowercased, stripped of punctuation and tokenized with word_tokenize; English stop-words are then removed.

Lemmatization (WordNetLemmatizer) reduces words to dictionary forms while Porter stemming and Lancaster stemming reduce them to root forms for comparison.

A Multinomial Naive Bayes text classifier built from scratch (log priors plus Laplace-smoothed log likelihoods) is trained on six labelled sentences across Technology, Sports and Health.

The Flask page has two parts: paragraph analysis (per-sentence pipeline table plus predicted category) and single-sentence classification with all processing stages shown.

Code

```
from flask import Flask, request, render_template_string
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.stem import WordNetLemmatizer, PorterStemmer, LancasterStemmer
import math
import string
import re

# --- NLTK Downloads (Runs once) ---
try:
    nltk.data.find('tokenizers/punkt')
    nltk.data.find('tokenizers/punkt_tab')
    nltk.data.find('corpora/stopwords')
    nltk.data.find('corpora/wordnet')
```

```
except LookupError:
    print("Downloading required NLTK data...")
    nltk.download('punkt')
    nltk.download('punkt_tab')  # Required for newer NLTK versions
    nltk.download('stopwords')
    nltk.download('wordnet')
    nltk.download('omw-1.4')    # Required for wordnet lemmatizer in some environments

app = Flask(__name__)

# ========================================
# 1. Text Preprocessing Pipeline
# ========================================
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
porter = PorterStemmer()
lancaster = LancasterStemmer()

def analyze_sentence(sentence):
    """Tokenize, remove stop-words, lemmatize, Porter-stem and Lancaster-stem."""
    # Clean and tokenize
    text = sentence.lower()
    text = re.sub(f"[{re.escape(string.punctuation)}]", "", text)
    tokens = word_tokenize(text)

    # 1. Raw tokens
    token_str = " ".join(tokens)

    # 2. Stop-word Removal
    filtered_words = [w for w in tokens if w not in stop_words]

    # 3. Lemmatization (dictionary form)
    lemmatized_words = [lemmatizer.lemmatize(w) for w in filtered_words]

    # 4. Porter Stemming
    porter_words = [porter.stem(w) for w in filtered_words]

    # 5. Lancaster Stemming (more aggressive)
    lancaster_words = [lancaster.stem(w) for w in filtered_words]

    return {
        'original': sentence,
        'tokens': token_str,
        'filtered': " ".join(filtered_words),
        'lemmatized': " ".join(lemmatized_words),
        'porter': " ".join(porter_words),
        'lancaster': " ".join(lancaster_words),
        'final_tokens': porter_words  # classifier input: stop-free Porter stems
    }

# ========================================
# 2. Naive Bayes Text Classifier (From Scratch)
# ========================================
```

```
class TextNaiveBayes:
    def __init__(self):
        self.vocab = set()
        self.class_word_counts = {}
        self.class_counts = {}
        self.total_docs = 0

    def fit(self, X_train, y_train):
        for text_tokens, label in zip(X_train, y_train):
            self.total_docs += 1
            if label not in self.class_counts:
                self.class_counts[label] = 0
                self.class_word_counts[label] = {}

            self.class_counts[label] += 1

            for word in text_tokens:
                self.vocab.add(word)
                self.class_word_counts[label][word] = self.class_word_counts[label].get(word, 0) + 1

    def predict(self, text_tokens):
        best_label = None
        max_prob = -float('inf')

        for label in self.class_counts:
            # P(Class) - Prior
            prob = math.log(self.class_counts[label] / self.total_docs)

            total_words_in_class = sum(self.class_word_counts[label].values())
            vocab_size = len(self.vocab)

            # P(Word | Class) - Likelihood with Laplace Smoothing
            for word in text_tokens:
                count = self.class_word_counts[label].get(word, 0)
                prob += math.log((count + 1) / (total_words_in_class + vocab_size))

            if prob > max_prob:
                max_prob = prob
                best_label = label

        return best_label

# ==========================================
# 3. Training the Model with Dummy Data
# ==========================================
training_data = [
    ("The new smartphone has a great camera and battery life.", "Technology"),
    ("Artificial intelligence is transforming software development.", "Technology"),
    ("The quarterback threw a touchdown pass in the final quarter.", "Sports"),
    ("The team won the championship after a grueling match.", "Sports"),
    ("Eating vegetables and exercising daily improves heart health.", "Health"),
    ("The vaccine has significantly reduced the spread of the virus.", "Health")
]
```

```
classifier = TextNaiveBayes()
X_train_processed = [analyze_sentence(text)['final_tokens'] for text, label in training_data]
y_train = [label for text, label in training_data]
classifier.fit(X_train_processed, y_train)

# ==========================================
# 4. Flask HTML Template & Routing
# ==========================================
HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
  <title>Pattern Analysis - NLP</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: #f4f7f6; margin: 0;
padding: 30px; }
    .container { max-width: 1150px; margin: auto; }
    .card { background: white; padding: 25px; border-radius: 10px; box-shadow: 0 5px 15px rgba(0,0,0,0.08);
margin-bottom: 30px; border-left: 5px solid #34495e; }
    h1, h2 { color: #2c3e50; }
    textarea, input[type="text"] { width: 100%; padding: 10px; border: 1px solid #ccc; border-radius: 4px;
box-sizing: border-box; margin-bottom: 15px; font-family: inherit;}
    button { background: #34495e; color: white; border: none; padding: 10px 20px; cursor: pointer;
border-radius: 5px; font-weight: bold; }
    button:hover { background: #2c3e50; }
    table { width: 100%; border-collapse: collapse; margin-top: 15px; font-size: 0.88em; }
    th, td { border: 1px solid #ddd; padding: 10px; text-align: left; vertical-align: top; }
    th { background: #f9f9f9; color: #333;}
    .category-badge { background: #e74c3c; color: white; padding: 4px 8px; border-radius: 4px; font-weight:
bold; white-space: nowrap; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Experiment 5: Text-Based Pattern Analysis</h1>

    <!-- PART 1: Paragraph Analysis -->
    <div class="card">
      <h2>Part 1: Paragraph Analysis & Classification</h2>
      <p>Enter a paragraph. The system extracts sentences, applies stop-word removal, lemmatization,
      Porter + Lancaster stemming, then classifies each sentence with Naive Bayes.</p>
      <form method="POST">
        <input type="hidden" name="form_type" value="paragraph">
        <textarea name="paragraph_text" rows="4" required>{{ saved_para if saved_para else "The software
update improved the phone's performance. Meanwhile, the striker scored a beautiful goal for the club. Doctors
recommend drinking plenty of water." }}</textarea>
        <button type="submit">Analyze Paragraph</button>
      </form>

      {% if para_results %}
      <table>
        <tr>
```

```
            <th>Original Sentence</th>
            <th>Tokens</th>
            <th>Stop-words Removed</th>
            <th>Lemmatized</th>
            <th>Porter Stem</th>
            <th>Lancaster Stem</th>
            <th>Predicted Category</th>
          </tr>
          {% for res in para_results %}
          <tr>
            <td>{{ res.original }}</td>
            <td style="color: #7f8c8d;">{{ res.tokens }}</td>
            <td style="color: #7f8c8d;">{{ res.filtered }}</td>
            <td style="color: #2980b9;">{{ res.lemmatized }}</td>
            <td style="color: #27ae60;">{{ res.porter }}</td>
            <td style="color: #8e44ad;">{{ res.lancaster }}</td>
            <td><span class="category-badge">{{ res.category }}</span></td>
          </tr>
          {% endfor %}
        </table>
        {% endif %}
      </div>

      <!-- PART 2: Single Sentence Prediction -->
      <div class="card" style="border-left-color: #e67e22;">
        <h2>Part 2: Single Sentence Classification</h2>
        <form method="POST">
          <input type="hidden" name="form_type" value="sentence">
          <input type="text" name="single_sentence" value="{{ saved_sent if saved_sent else 'The basketball
player injured his ankle.' }}" required>
          <button type="submit" style="background: #e67e22;">Predict Category</button>
        </form>

        {% if single_result %}
        <div style="background: #fef5e7; padding: 15px; border-radius: 8px; margin-top: 15px; border: 1px solid
#fad7a1;">
          <p><strong>Input:</strong> {{ single_result.original }}</p>
          <p><strong>Tokens:</strong> {{ single_result.tokens }}</p>
          <p><strong>Stop-words removed:</strong> {{ single_result.filtered }}</p>
          <p><strong>Lemmatized:</strong> {{ single_result.lemmatized }}</p>
          <p><strong>Porter stem:</strong> {{ single_result.porter }}</p>
          <p><strong>Lancaster stem:</strong> {{ single_result.lancaster }}</p>
          <p><strong>Predicted Category:</strong> <span class="category-badge"
style="background:#d35400;">{{ single_result.category }}</span></p>
        </div>
        {% endif %}
      </div>
    </div>
</body>
</html>
"""

@app.route('/', methods=['GET', 'POST'])
```

```
def index():
    para_results = None
    single_result = None
    saved_para = ""
    saved_sent = ""

    if request.method == 'POST':
        form_type = request.form.get('form_type')

        if form_type == 'paragraph':
            text = request.form.get('paragraph_text')
            saved_para = text
            # Split paragraph into individual sentences
            sentences = sent_tokenize(text)

            para_results = []
            for sentence in sentences:
                analyzed = analyze_sentence(sentence)
                category = classifier.predict(analyzed['final_tokens'])
                analyzed['category'] = category
                para_results.append(analyzed)

        elif form_type == 'sentence':
            sentence = request.form.get('single_sentence')
            saved_sent = sentence
            analyzed = analyze_sentence(sentence)
            category = classifier.predict(analyzed['final_tokens'])
            analyzed['category'] = category
            single_result = analyzed

    return render_template_string(HTML_TEMPLATE,
                                  para_results=para_results,
                                  single_result=single_result,
                                  saved_para=saved_para,
                                  saved_sent=saved_sent)

if __name__ == '__main__':
    print("Starting Text Analysis App...")
    print("Access at: http://127.0.0.1:5000")
    app.run(debug=True)
```

Output

Experiment 5: Text-Based Pattern Analysis

Part 1: Paragraph Analysis & Classification

Enter a paragraph. The system extracts sentences, applies stop-word removal, lemmatization, Porter + Lancaster stemming, then classifies each sentence with Naive Bayes.

The software update improved the phone's performance. Meanwhile, the striker scored a beautiful goal for the club. Doctors recommend drinking plenty of water.

Analyze Paragraph

Original Sentence	Tokens	Stop-words Removed	Lemmatized	Porter Stem	Lancaster Stem	Predicted Category
The software update improved the phone's performance.	the software update improved the phones performance	software update improved phones performance	software update improved phone performance	softwar updat improv phone perform	softw upd improv phon perform	Technology
Meanwhile, the striker scored a beautiful goal for the club.	meanwhile the striker scored a beautiful goal for the club	meanwhile striker scored beautiful goal club	meanwhile striker scored beautiful goal club	meanwhil striker score beauti goal club	meanwhil striker scor beauty goal club	Sports
Doctors recommend drinking plenty of water.	doctors recommend drinking plenty of water	doctors recommend drinking plenty water	doctor recommend drinking plenty water	doctor recommend drink plenti water	doct recommend drink plenty wat	Sports

Part 2: Single Sentence Classification

The basketball player injured his ankle.

Predict Category

Input: The basketball player injured his ankle.

Tokens: the basketball player injured his ankle

Stop-words removed: basketball player injured ankle

Lemmatized: basketball player injured ankle

Porter stem: basketbal player injur ankl

Lancaster stem: basketbal play ind ankl

Predicted Category: Sports

Experiment 6

Objective

Prepare a page using flask consisting of a code in python to perform Bayesian Risk Minimization for at least three classes, three actions. Given loss function, particular action should trigger particular action, that action should reflect in the page itself.

Theory

Three machine states (Normal, Warning, Danger) with prior probabilities and a temperature sensor reading (Cool, Moderate, High, Critical) with class-conditional likelihoods form the Bayesian setup.

Posterior probabilities P(Class|Reading) are computed with Bayes theorem for the observed sensor value.

A 3x3 loss matrix gives the cost of each action (Assume Normal, Assume Warning, Assume Danger) under each true class; missing a Danger state costs 1000.

Expected risk R(action|x) is the loss weighted by the posteriors, and the minimum-risk action is selected - this need not be the most probable class (e.g. a Moderate reading selects Assume Warning).

The selected action triggers its mapped response on the page itself: a green continue banner, a yellow maintenance alert, or a red emergency shutdown banner.

Code

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

# --- Bayesian Configuration ---

classes = ['Normal', 'Warning', 'Danger']

# Prior Probabilities P(C)
priors = {
   'Normal': 0.60,
   'Warning': 0.30,
   'Danger': 0.10
}

# Likelihoods P(X | C)
likelihoods = {
```

```
    'Cool':     {'Normal': 0.60, 'Warning': 0.10, 'Danger': 0.00},
    'Moderate': {'Normal': 0.30, 'Warning': 0.40, 'Danger': 0.10},
    'High':     {'Normal': 0.09, 'Warning': 0.40, 'Danger': 0.30},
    'Critical': {'Normal': 0.01, 'Warning': 0.10, 'Danger': 0.60}
}

# Loss Matrix: loss_matrix[action][true_class]
loss_matrix = {
    'Assume Normal':  {'Normal': 0,   'Warning': 50, 'Danger': 1000},
    'Assume Warning': {'Normal': 20,  'Warning': 0,  'Danger': 200},
    'Assume Danger':  {'Normal': 100, 'Warning': 50, 'Danger': 0}
}

# Action Triggers based on Selected Minimum Risk Class
action_triggers = {
    'Assume Normal':  "🟢 Continue Normal Operation.",
    'Assume Warning': "🟡 Trigger: Issue Maintenance Alert - Schedule check within 24h.",
    'Assume Danger':  "🔴 Trigger: EMERGENCY SHUTDOWN - Halt production immediately!"
}

# --- HTML Template ---
HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>Bayesian Risk Minimization</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 800px; margin: 40px auto; padding: 20px; background:
#f9f9f9; }
        .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 4px 6px rgba(0,0,0,0.1);
margin-bottom: 20px; }
        h1, h2 { color: #333; }
        table { width: 100%; border-collapse: collapse; margin-top: 10px; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align: center; }
        th { background-color: #f2f2f2; }
        .action-box { font-size: 1.2em; font-weight: bold; padding: 15px; border-radius: 5px; color: white; text-align:
center;}
        .Normal { background: #2ecc71; }
        .Warning { background: #f1c40f; color: black; }
        .Danger { background: #e74c3c; }
        .note { font-size: 0.9em; color: #555; font-style: italic; }
    </style>
</head>
<body>
    <h1>Industrial Sensor Risk Minimization</h1>

    <div class="card">
        <h2>0. Problem Setup</h2>
        <p><b>Classes:</b> {% for c in classes %}{{ c }}{% if not loop.last %}, {% endif %}{% endfor %}</p>
        <p><b>Actions:</b> {% for a in actions %}{{ a }}{% if not loop.last %}, {% endif %}{% endfor %}</p>
        <h2 style="font-size:1.1em;">Prior Probabilities P(C)</h2>
        <table>
            <tr><th>Class</th><th>Prior</th></tr>
```

```
      {% for c in classes %}
      <tr><td>{{ c }}</td><td>{{ priors[c] }}</td></tr>
      {% endfor %}
    </table>
    <h2 style="font-size:1.1em;">Likelihoods P(X | C)</h2>
    <table>
      <tr><th>Sensor Reading (X)</th>{% for c in classes %}<th>{{ c }}</th>{% endfor %}</tr>
      {% for x in readings %}
      <tr><td>{{ x }}</td>{% for c in classes %}<td>{{ likelihoods[x][c] }}</td>{% endfor %}</tr>
      {% endfor %}
    </table>
    <h2 style="font-size:1.1em;">Loss Matrix &amp;lambda;(Action | True Class)</h2>
    <table>
      <tr><th>Action / True Class</th>{% for c in classes %}<th>{{ c }}</th>{% endfor %}</tr>
      {% for a in actions %}
      <tr><td><b>{{ a }}</b></td>{% for c in classes %}<td>{{ loss_matrix[a][c] }}</td>{% endfor %}</tr>
      {% endfor %}
    </table>
  </div>

  <div class="card">
    <form method="POST">
      <label for="sensor_reading"><b>Select Temperature Sensor Reading:</b></label>
      <select name="sensor_reading" id="sensor_reading" style="padding:5px; margin-left:10px;">
        <option value="Cool">Cool</option>
        <option value="Moderate">Moderate (Demonstrates Risk vs Probability)</option>
        <option value="High">High</option>
        <option value="Critical">Critical</option>
      </select>
      <button type="submit" style="padding:6px 15px; margin-left:10px; cursor:pointer;">Analyze</button>
    </form>
  </div>

  {% if results %}
  <div class="card">
    <h2>1. Input</h2>
    <p>Observed Sensor Reading: <b>{{ results.input }}</b></p>
  </div>

  <div class="card">
    <h2>2. Posterior Probabilities $P(Class | Input)$</h2>
    <table>
      <tr><th>Class</th><th>Calculated Probability</th></tr>
      {% for cls, prob in results.posteriors.items() %}
      <tr><td>{{ cls }}</td><td>{{ "%.1f"|format(prob * 100) }}%</td></tr>
      {% endfor %}
    </table>
    <p class="note">Notice: Highest probability does not always mean it's the safest choice.</p>
  </div>

  <div class="card">
    <h2>3. Expected Risk (Loss) for Each Action</h2>
    <table>
```

```
        <tr><th>Action Considered</th><th>Expected Risk Score</th></tr>
        {% for action, risk in results.risks.items() %}
        <tr {% if action == results.selected_action %}style="background-color:#e8f4f8; font-weight:bold;"{% endif
%}>
          <td>{{ action }}</td>
          <td>{{ "%.2f"|format(risk) }}</td>
        </tr>
        {% endfor %}
      </table>
    </div>

    <div class="card">
      <h2>4. Selected Decision & Triggered Action</h2>
      <p>Selected Class Action (Minimum Risk): <b>{{ results.selected_action }}</b></p>

      <div class="action-box {{ results.style }}">
        {{ results.triggered_action }}
      </div>
    </div>
    {% endif %}

</body>
</html>
"""

@app.route('/', methods=['GET', 'POST'])
def index():
    results = None
    if request.method == 'POST':
        x = request.form['sensor_reading']

        # 1. Calculate Evidence P(X) to normalize probabilities
        px = sum([likelihoods[x][c] * priors[c] for c in classes])

        # 2. Calculate Posterior Probabilities P(C|X)
        posteriors = {}
        for c in classes:
            posteriors[c] = (likelihoods[x][c] * priors[c]) / px

        # 3. Calculate Expected Risk for each possible action
        # Risk = Sum( Loss(Action | TrueClass) * P(TrueClass | X) )
        risks = {}
        for action in loss_matrix.keys():
            expected_risk = 0
            for c in classes:
                expected_risk += loss_matrix[action][c] * posteriors[c]
            risks[action] = expected_risk

        # 4. Select Action with MINIMUM Risk
        selected_action = min(risks, key=risks.get)

        # Map styling
        style_map = {'Assume Normal': 'Normal', 'Assume Warning': 'Warning', 'Assume Danger': 'Danger'}
```

```
        results = {
            'input': x,
            'posteriors': posteriors,
            'risks': risks,
            'selected_action': selected_action,
            'triggered_action': action_triggers[selected_action],
            'style': style_map[selected_action]
        }

    return render_template_string(HTML_TEMPLATE, results=results,
                                  classes=classes, actions=list(loss_matrix.keys()),
                                  priors=priors, likelihoods=likelihoods,
                                  loss_matrix=loss_matrix,
                                  readings=list(likelihoods.keys()))

if __name__ == '__main__':
    print("Server running at http://127.0.0.1:5000/")
    app.run(debug=True)
```

Output

Industrial Sensor Risk Minimization

0. Problem Setup

Classes: Normal, Warning, Danger

Actions: Assume Normal, Assume Warning, Assume Danger

Prior Probabilities P(C)

Class	Prior
Normal	0.6
Warning	0.3
Danger	0.1

Likelihoods P(X | C)

Sensor Reading (X)	Normal	Warning	Danger
Cool	0.6	0.1	0.0
Moderate	0.3	0.4	0.1
High	0.09	0.4	0.3
Critical	0.01	0.1	0.6

Loss Matrix λ(Action | True Class)

Action / True Class	Normal	Warning	Danger
Assume Normal	0	50	1000
Assume Warning	20	0	200
Assume Danger	100	50	0

Select Temperature Sensor Reading: Cool Analyze

1. Input

Observed Sensor Reading: **Cool**

2. Posterior Probabilities $P(Class | Input)$

Class	Calculated Probability
Normal	92.3%
Warning	7.7%
Danger	0.0%

Notice: Highest probability does not always mean it's the safest choice.

3. Expected Risk (Loss) for Each Action

Action Considered	Expected Risk Score
Assume Normal	**3.85**
Assume Warning	18.46
Assume Danger	96.15

4. Selected Decision & Triggered Action

Selected Class Action (Minimum Risk): **Assume Normal**

Continue Normal Operation.

Experiment 7

Objective

Implement Linear regression in python (no inbuilt function should be used) to find optimized parameters using Gradient descent for any application and Plot Gradient Descent curve.

Theory

Univariate linear regression models salary as y = mX + c for years of experience, with a synthetic dataset (base 20000 + 5000 per year plus noise).

Features are standardized (zero mean, unit variance) so gradient descent converges in 100 epochs with learning rate 0.1.

Mean Squared Error J(m, c) = (1/2n) times the sum of squared errors is minimized by iteratively stepping m and c against their gradients.

Gradients dm = (1/n) times sum((y_pred - y)X) and dc = (1/n) times sum(y_pred - y) are computed from scratch with numpy only.

Two plots are produced and saved: the fitted regression line over the data and the gradient descent cost-vs-epoch curve showing convergence.

Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# ==========================================
# 1. Dataset Generation
# ==========================================
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=float)
y = 5000 * X + 20000 + np.random.normal(0, 3000, len(X))

# --- THE FIX: Feature Scaling (Standardization) ---
# This transforms X to have a mean of 0 and a standard deviation of 1.
X_mean = np.mean(X)
X_std = np.std(X)
X_scaled = (X - X_mean) / X_std

# ==========================================
# 2. Gradient Descent Algorithm
```

```
# ==========================================
def linear_regression_gd(X_data, y_data, learning_rate=0.1, epochs=100):
    n = len(X_data)
    m = 0.0
    c = 0.0
    cost_history = []

    for i in range(epochs):
        y_pred = m * X_data + c

        cost = (1 / (2 * n)) * np.sum((y_pred - y_data) ** 2)
        cost_history.append(cost)

        dm = (1 / n) * np.sum((y_pred - y_data) * X_data)
        dc = (1 / n) * np.sum(y_pred - y_data)

        m -= learning_rate * dm
        c -= learning_rate * dc

    return m, c, cost_history

# ==========================================
# 3. Train the Model
# ==========================================
# Notice we now need far fewer epochs (100) and can use a higher learning rate (0.1)
m_opt, c_opt, cost_history = linear_regression_gd(X_scaled, y, learning_rate=0.1, epochs=100)

# ==========================================
# 4. Plotting the Results
# ==========================================
plt.figure(figsize=(14, 5))

# Plot 1: The Regression Line vs Actual Data
plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='Actual Salaries')

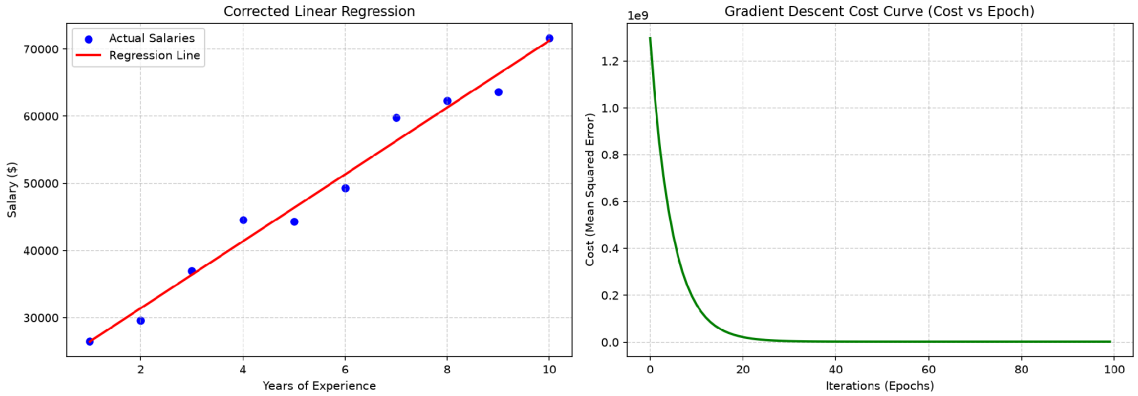
# We calculate the line using the scaled features, but plot it against the original X
plt.plot(X, m_opt * X_scaled + c_opt, color='red', linewidth=2, label='Regression Line')
plt.title('Corrected Linear Regression')
plt.xlabel('Years of Experience')
plt.ylabel('Salary ($)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

# Plot 2: Gradient Descent Optimization Curve
plt.subplot(1, 2, 2)
plt.plot(range(100), cost_history, color='green', linewidth=2)
plt.title('Gradient Descent Cost Curve (Cost vs Epoch)')
plt.xlabel('Iterations (Epochs)')
plt.ylabel('Cost (Mean Squared Error)')
plt.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
```

```
import os
out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                        "linear_regression.png")
plt.savefig(out_path, dpi=150)
print(f"Plot saved to {out_path}")
plt.show()
```

Output



Experiment 8

Objective

Implement Logistic regression for binary class and multiclass classification in python (no inbuilt function should be used) to find optimized parameters using Gradient descent for any application and Plot Gradient Descent curve.

Theory

Binary logistic regression uses the sigmoid activation with binary cross-entropy loss; multiclass logistic regression uses the softmax activation with categorical cross-entropy loss - both optimized by gradient descent from scratch with numpy only.

The application uses synthetic 2D clustered data: classes 0 and 1 for the binary model, classes 0, 1 and 2 for the multiclass model, both standardized before training.

Gradients dW = (1/m)X'(A - Y) and db = (1/m)sum(A - Y) update the weights and bias over 500 epochs at learning rate 0.1.

Two gradient descent cost curves are plotted and saved: binary log-loss (0.693 to 0.031, 99.5% accuracy) and multiclass cross-entropy (1.099 to 0.063, 99.3% accuracy).

Code

```
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# 1. Binary Logistic Regression
# ========================================
class BinaryLogisticRegression:
    def __init__(self, learning_rate=0.05, epochs=1000):
        self.lr = learning_rate
        self.epochs = epochs
        self.cost_history = []

    def sigmoid(self, z):
        # Clip z to prevent overflow in exp
        z = np.clip(z, -250, 250)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        m, n = X.shape
```

```
        self.W = np.zeros((n, 1))
        self.b = 0

        # Ensure y is a column vector
        y = y.reshape(-1, 1)

        for i in range(self.epochs):
            # 1. Linear combination & Activation
            Z = np.dot(X, self.W) + self.b
            A = self.sigmoid(Z)

            # 2. Cost Calculation (Binary Cross-Entropy)
            # Add small epsilon to prevent log(0)
            epsilon = 1e-15
            cost = -(1/m) * np.sum(y * np.log(A + epsilon) + (1 - y) * np.log(1 - A + epsilon))
            self.cost_history.append(cost)

            # 3. Gradients
            dW = (1/m) * np.dot(X.T, (A - y))
            db = (1/m) * np.sum(A - y)

            # 4. Update parameters
            self.W -= self.lr * dW
            self.b -= self.lr * db

    def predict(self, X):
        Z = np.dot(X, self.W) + self.b
        return (self.sigmoid(Z) >= 0.5).astype(int)

# ========================================
# 2. Multiclass Logistic Regression (Softmax)
# ========================================
class MulticlassLogisticRegression:
    def __init__(self, learning_rate=0.05, epochs=1000):
        self.lr = learning_rate
        self.epochs = epochs
        self.cost_history = []

    def softmax(self, z):
        # Shift z for numerical stability
        exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)

    def to_one_hot(self, y, num_classes):
        one_hot = np.zeros((len(y), num_classes))
        one_hot[np.arange(len(y)), y] = 1
        return one_hot

    def fit(self, X, y):
        m, n = X.shape
        num_classes = len(np.unique(y))
        y_oh = self.to_one_hot(y, num_classes)
```

```
        # W shape: (n_features, n_classes), b shape: (1, n_classes)
        self.W = np.zeros((n, num_classes))
        self.b = np.zeros((1, num_classes))

        for i in range(self.epochs):
            # 1. Linear combination & Softmax Activation
            Z = np.dot(X, self.W) + self.b
            A = self.softmax(Z)

            # 2. Cost Calculation (Categorical Cross-Entropy)
            epsilon = 1e-15
            cost = -(1/m) * np.sum(y_oh * np.log(A + epsilon))
            self.cost_history.append(cost)

            # 3. Gradients
            dW = (1/m) * np.dot(X.T, (A - y_oh))
            db = (1/m) * np.sum(A - y_oh, axis=0, keepdims=True)

            # 4. Update parameters
            self.W -= self.lr * dW
            self.b -= self.lr * db

    def predict(self, X):
        Z = np.dot(X, self.W) + self.b
        A = self.softmax(Z)
        return np.argmax(A, axis=1)

# ========================================
# 3. Application / Execution
# ========================================
np.random.seed(42)

# Generate Synthetic Data: 3 Distinct Clusters
# Class 0: centered at (2,2), Class 1: (6,6), Class 2: (2,8)
X_0 = np.random.randn(100, 2) + np.array([2, 2])
X_1 = np.random.randn(100, 2) + np.array([6, 6])
X_2 = np.random.randn(100, 2) + np.array([2, 8])

# Binary Dataset (Classes 0 and 1 only)
X_bin = np.vstack([X_0, X_1])
y_bin = np.hstack([np.zeros(100), np.ones(100)]).astype(int)

# Multiclass Dataset (Classes 0, 1, and 2)
X_multi = np.vstack([X_0, X_1, X_2])
y_multi = np.hstack([np.zeros(100), np.ones(100), np.full(100, 2)]).astype(int)

# Standardizing the Binary Dataset
X_bin_mean = np.mean(X_bin, axis=0)
X_bin_std = np.std(X_bin, axis=0)
X_bin_scaled = (X_bin - X_bin_mean) / X_bin_std

# Standardizing the Multiclass Dataset
X_multi_mean = np.mean(X_multi, axis=0)
```

```
X_multi_std = np.std(X_multi, axis=0)
X_multi_scaled = (X_multi - X_multi_mean) / X_multi_std

# Then pass the scaled data to the fit functions:
# Train Binary Model
binary_model = BinaryLogisticRegression(learning_rate=0.1, epochs=500)
#binary_model.fit(X_bin, y_bin)
binary_model.fit(X_bin_scaled, y_bin)

# Train Multiclass Model
multi_model = MulticlassLogisticRegression(learning_rate=0.1, epochs=500)
#multi_model.fit(X_multi, y_multi)
multi_model.fit(X_multi_scaled, y_multi)

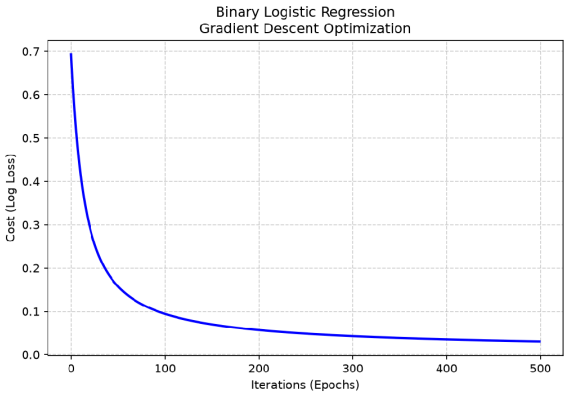
# ========================================
# 4. Plotting the Gradient Descent Curves
# ========================================
plt.figure(figsize=(14, 5))

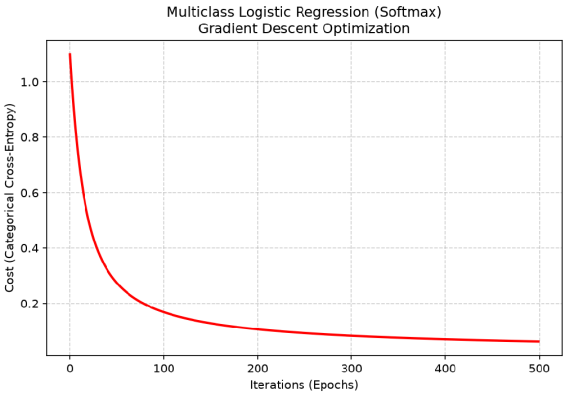
# Plot 1: Binary LR Cost Curve
plt.subplot(1, 2, 1)
plt.plot(range(binary_model.epochs), binary_model.cost_history, color='blue', linewidth=2)
plt.title('Binary Logistic Regression\nGradient Descent Optimization')
plt.xlabel('Iterations (Epochs)')
plt.ylabel('Cost (Log Loss)')
plt.grid(True, linestyle='--', alpha=0.6)

# Plot 2: Multiclass LR Cost Curve
plt.subplot(1, 2, 2)
plt.plot(range(multi_model.epochs), multi_model.cost_history, color='red', linewidth=2)
plt.title('Multiclass Logistic Regression (Softmax)\nGradient Descent Optimization')
plt.xlabel('Iterations (Epochs)')
plt.ylabel('Cost (Categorical Cross-Entropy)')
plt.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
import os
out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                        "logistic_regression.png")
plt.savefig(out_path, dpi=150)
print(f"Plot saved to {out_path}")
plt.show()
```

Output


Binary Logistic Regression
Gradient Descent Optimization
Cost (Log Loss)
Iterations (Epochs)
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
100
200
300
400
500


Multiclass Logistic Regression (Softmax)
Gradient Descent Optimization
Cost (Categorical Cross-Entropy)
Iterations (Epochs)
0.2
0.4
0.6
0.8
1.0
0
100
200
300
400
500

Experiment 9

Objective

Implement regularization for any application show plots of 1) decision boundary and 2) performance before and after applying regularization.

Theory

Logistic regression with L2 regularization is built from scratch: cost = binary cross-entropy + (lambda/2m) times the squared weight norm, with the gradient gaining a (lambda/m)W penalty term (bias unpenalized).

The application uses concentric circles (inner blob vs outer ring with label noise) expanded to degree-8 polynomial features, where the unregularized model memorizes noise.

Without regularization the model reaches 100% train vs 90% test accuracy (10% gap, weight norm 57, wiggly boundary); with L2 (lambda = 0.5) the gap halves to 4.8% with weight norm 12 and a smooth boundary.

Two plots are produced and saved: decision boundaries before vs after regularization and grouped train/test accuracy bars showing performance before vs after.

Code

```
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# 1. Application dataset (from scratch)
# Concentric circles: inner blob = class 0, outer ring = class 1.
# Small training set + high-degree polynomial features => overfits
# without regularization.
# ========================================
np.random.seed(11)

def make_circles(n_inner=25, n_ring=25, noise=0.6, flip=0.05):
    r_inner = np.abs(np.random.randn(n_inner)) * 0.9
    t_inner = np.random.rand(n_inner) * 2 * np.pi
    X0 = np.c_[r_inner * np.cos(t_inner), r_inner * np.sin(t_inner)]
    r_ring = 2.5 + np.random.randn(n_ring) * noise
    t_ring = np.random.rand(n_ring) * 2 * np.pi
    X1 = np.c_[r_ring * np.cos(t_ring), r_ring * np.sin(t_ring)]
    X = np.vstack([X0, X1])
```

```
    y = np.hstack([np.zeros(n_inner), np.ones(n_ring)]).astype(int)
    noisy = np.random.rand(len(y)) < flip  # label noise: forces memorization
    y[noisy] = 1 - y[noisy]
    return X, y

X_train, y_train = make_circles()
X_test, y_test = make_circles(n_inner=60, n_ring=60)

# Polynomial feature expansion (degree 8, no inbuilt function)
DEGREE = 8

def poly_features(X):
    x1, x2 = X[:, 0], X[:, 1]
    cols = [np.ones(len(X))]
    for d in range(1, DEGREE + 1):
        for k in range(d + 1):
            cols.append((x1 ** (d - k)) * (x2 ** k))
    return np.column_stack(cols)

Phi_train = poly_features(X_train)
Phi_test = poly_features(X_test)

# Standardize using training statistics (skip bias column)
mu = Phi_train[:, 1:].mean(axis=0)
sd = Phi_train[:, 1:].std(axis=0) + 1e-12
Phi_train[:, 1:] = (Phi_train[:, 1:] - mu) / sd
Phi_test[:, 1:] = (Phi_test[:, 1:] - mu) / sd

# ============================================
# 2. Logistic Regression with L2 (from scratch)
# Cost = BCE + (lambd/2m) * ||W||^2 ; gradient gets + (lambd/m)*W
# ============================================
class LogisticL2:
    def __init__(self, learning_rate=0.5, epochs=3000, lambd=0.0):
        self.lr = learning_rate
        self.epochs = epochs
        self.lambd = lambd
        self.cost_history = []

    def sigmoid(self, z):
        z = np.clip(z, -250, 250)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.W = np.zeros((n, 1))
        self.b = 0.0
        for _ in range(self.epochs):
            A = self.sigmoid(X @ self.W + self.b)
            eps = 1e-15
            bce = -(1 / m) * np.sum(y * np.log(A + eps) + (1 - y) * np.log(1 - A + eps))
            reg = (self.lambd / (2 * m)) * np.sum(self.W[1:] ** 2)  # don't penalize bias
```

```
            self.cost_history.append(bce + reg)
            dW = (1 / m) * (X.T @ (A - y))
            dW[1:] += (self.lambd / m) * self.W[1:]
            db = (1 / m) * np.sum(A - y)
            self.W -= self.lr * dW
            self.b -= self.lr * db
        return self

    def predict(self, X):
        return (self.sigmoid(X @ self.W + self.b) >= 0.5).astype(int).ravel()

    def accuracy(self, X, y):
        return float(np.mean(self.predict(X) == y) * 100)

# ==========================================
# 3. Train before (lambd=0) vs after (lambd>0) regularization
# ==========================================
model_plain = LogisticL2(lambd=0.0).fit(Phi_train, y_train)
model_reg = LogisticL2(lambd=0.5).fit(Phi_train, y_train)

acc = {
    "plain_train": model_plain.accuracy(Phi_train, y_train),
    "plain_test": model_plain.accuracy(Phi_test, y_test),
    "reg_train": model_reg.accuracy(Phi_train, y_train),
    "reg_test": model_reg.accuracy(Phi_test, y_test),
}
w_plain = float(np.abs(model_plain.W[1:]).sum())
w_reg = float(np.abs(model_reg.W[1:]).sum())
print(f"Without regularization : train={acc['plain_train']:.1f}%  test={acc['plain_test']:.1f}%  |W|={w_plain:.0f}")
print(f"With L2 (lambd=0.5)    : train={acc['reg_train']:.1f}%  test={acc['reg_test']:.1f}%  |W|={w_reg:.0f}")
print(f"Overfitting gap        : {acc['plain_train']-acc['plain_test']:.1f}% -> "
      f"{acc['reg_train']-acc['reg_test']:.1f}%")

# ==========================================
# 4. Plots: decision boundaries + performance before/after
# ==========================================
def boundary_grid(model, h=0.05):
    x_min, x_max = -4, 4
    y_min, y_max = -4, 4
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Phi = poly_features(np.c_[xx.ravel(), yy.ravel()])
    Phi[:, 1:] = (Phi[:, 1:] - mu) / sd
    return xx, yy, model.predict(Phi).reshape(xx.shape)

plt.figure(figsize=(18, 5))

plt.subplot(1, 3, 1)
xx, yy, Z = boundary_grid(model_plain)
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap="coolwarm", edgecolors="k")
plt.title(f"Decision Boundary WITHOUT Regularization\ntrain={acc['plain_train']:.1f}%
test={acc['plain_test']:.1f}%")
plt.xlabel("x1")
```

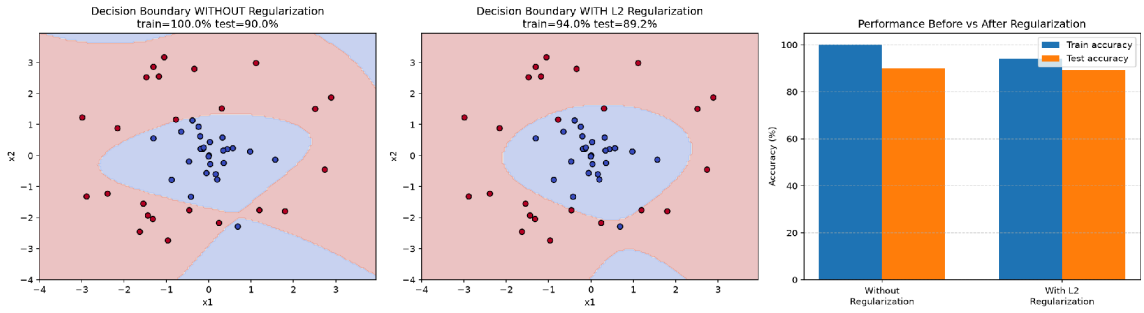
```
plt.ylabel("x2")

plt.subplot(1, 3, 2)
xx, yy, Z = boundary_grid(model_reg)
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap="coolwarm", edgecolors="k")
plt.title(f"Decision Boundary WITH L2 Regularization\ntrain={acc['reg_train']:.1f}% test={acc['reg_test']:.1f}%")
plt.xlabel("x1")
plt.ylabel("x2")

plt.subplot(1, 3, 3)
labels = ["Without\nRegularization", "With L2\nRegularization"]
x = np.arange(len(labels))
w = 0.35
plt.bar(x - w / 2, [acc["plain_train"], acc["reg_train"]], w, label="Train accuracy")
plt.bar(x + w / 2, [acc["plain_test"], acc["reg_test"]], w, label="Test accuracy")
plt.xticks(x, labels)
plt.ylabel("Accuracy (%)")
plt.ylim(0, 105)
plt.title("Performance Before vs After Regularization")
plt.legend()
plt.grid(True, axis="y", linestyle="--", alpha=0.6)

plt.tight_layout()
import os
out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                        "regularization.png")
plt.savefig(out_path, dpi=150)
print(f"Plot saved to {out_path}")
plt.show()
```

Output



Experiment 10

Objective

Implement Principal Component Analysis (PCA) for any application which involves testing. Also show the projected line and new reduced feature mapped on projected line.

Theory

The application uses study hours vs exam score (30 points, moderately correlated); features are standardized so both count equally in the covariance.

PCA fits on the standardized data: eigendecomposition of the covariance matrix gives PC1 (84.5% variance) as the projected line direction.

Every original point is projected onto the PC1 line as z = xs.pc1, giving the new 1D reduced feature, and mapped back to 2D with the inverse transform for display.

Two plots are produced and saved: original points with connector lines to their mapped points on the red PC1 line, and the explained variance bars (reconstruction MSE 8.66).

Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# ============================================
# 1. Application dataset
# Study hours (x1) vs exam score (x2), 30 points.
# ============================================
np.random.seed(11)

hours = np.random.uniform(1, 10, 30)
scores = 3 * hours + 20 + np.random.normal(0, 9, 30)
X = np.c_[hours, scores]

# Standardize (so both features count equally)
mu = X.mean(axis=0)
sd = X.std(axis=0)
Xs = (X - mu) / sd

# ========================================
# 2. PCA with sklearn
# ========================================
```

```
pca = PCA(n_components=1)
z = pca.fit_transform(Xs).ravel()            # new 1D reduced feature
pc1 = pca.components_[0]
var_ratio = pca.explained_variance_ratio_
X_proj = pca.inverse_transform(z.reshape(-1, 1)) * sd + mu  # mapped on PC1 line

print(f"PC1 direction: [{pc1[0]:.3f}, {pc1[1]:.3f}]")
print(f"Explained variance: PC1={var_ratio[0]*100:.1f}%")
mse = float(np.mean((X - X_proj) ** 2))
print(f"Reconstruction MSE (1 component): {mse:.2f}")

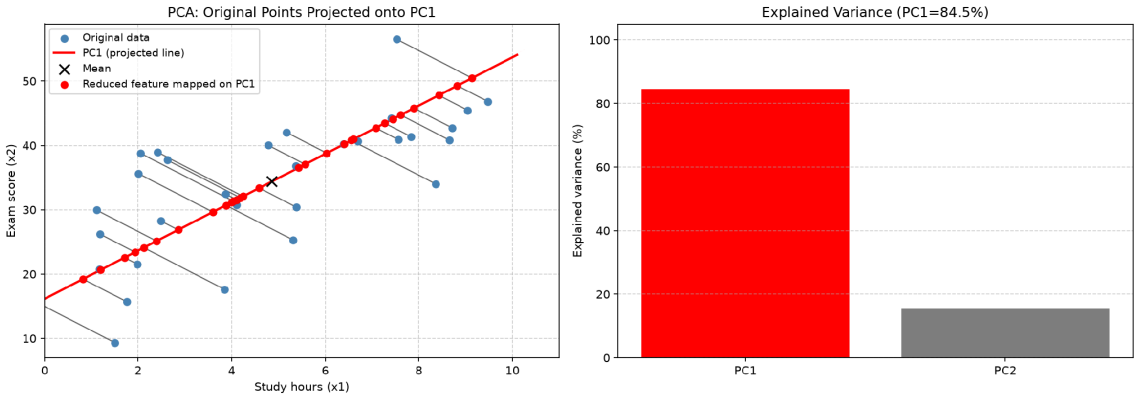
# ============================================
# 3. Plots: projection onto PC1 + explained variance
# ============================================
plt.figure(figsize=(14, 5))

# Plot 1: original points -> lines -> new points on the PC1 line
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c="steelblue", label="Original data", zorder=3)
d1 = pc1 * sd  # 1 step along PC1 in standardized space, expressed in original units
s = np.linspace(z.min() - 0.5, z.max() + 0.5, 100)
plt.plot(mu[0] + s * d1[0], mu[1] + s * d1[1], "r-", linewidth=2, label="PC1 (projected line)")
plt.scatter([mu[0]], [mu[1]], c="black", s=80, marker="x", label="Mean", zorder=5)
for p, q in zip(X, X_proj):  # projection line: original point -> new point on PC1
   plt.plot([p[0], q[0]], [p[1], q[1]], "k-", linewidth=1.0, alpha=0.6)
plt.scatter(X_proj[:, 0], X_proj[:, 1], c="red", s=35,
         label="Reduced feature mapped on PC1", zorder=4)
plt.title("PCA: Original Points Projected onto PC1")
plt.xlabel("Study hours (x1)")
plt.ylabel("Exam score (x2)")
plt.legend(fontsize=9)
plt.grid(True, linestyle="--", alpha=0.6)
plt.xlim(0, 11)

# Plot 2: explained variance
plt.subplot(1, 2, 2)
plt.bar(["PC1", "PC2"], [var_ratio[0] * 100, 100 - var_ratio[0] * 100],
      color=["red", "gray"])
plt.ylabel("Explained variance (%)")
plt.ylim(0, 105)
plt.title(f"Explained Variance (PC1={var_ratio[0]*100:.1f}%)")
plt.grid(True, axis="y", linestyle="--", alpha=0.6)

plt.tight_layout()
import os
out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "pca.png")
plt.savefig(out_path, dpi=150)
print(f"Plot saved to {out_path}")
plt.show()
```

Output


PCA: Original Points Projected onto PC1
Original data
PC1 (projected line)
Mean
Reduced feature mapped on PC1
Exam score (x2)
Study hours (x1)
0
2
4
6
8
10
10
20
30
40
50
Explained Variance (PC1=84.5%)
Explained variance (%)
0
20
40
60
80
100
PC1
PC2

Experiment 11

Objective

Implement Linear Discriminant Analysis (LDA) for any application which involves testing. Also show the projected line and new reduced feature mapped on projected line.

Theory

The application uses tumor size vs texture for benign vs malignant classification (80 train, 40 test points); unlike PCA, LDA uses class labels to maximize separation.

sklearn LinearDiscriminantAnalysis fits the discriminant direction on the training set; test points are projected with it and scored, giving 100% test accuracy.

Every point is dropped perpendicularly onto the red LDA projected line, showing the new 1D discriminant feature mapped back in 2D.

The saved plot shows the discriminant line, per-point projection connectors, mapped train/test points and the accuracy in the title.

Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# ========================================
# 1. Application dataset
# Tumor diagnosis: size (x1) vs texture (x2).
# Class 0 = benign, Class 1 = malignant. Train fits LDA, test is evaluated.
# ========================================
np.random.seed(5)

def blob(center, n, spread):
    return np.random.randn(n, 2) * spread + np.array(center)

X0_train, X1_train = blob([2, 2], 40, 1.2), blob([5, 6], 40, 1.2)
X0_test, X1_test = blob([2, 2], 20, 1.2), blob([5, 6], 20, 1.2)
X_train = np.vstack([X0_train, X1_train])
y_train = np.hstack([np.zeros(40), np.ones(40)]).astype(int)
X_test = np.vstack([X0_test, X1_test])
y_test = np.hstack([np.zeros(20), np.ones(20)]).astype(int)

# ========================================
# 2. LDA with sklearn (fit on TRAIN only)
```

```
# ========================================
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

w = lda.scalings_[:, 0] / np.linalg.norm(lda.scalings_[:, 0])  # LDA direction
if (X1_train @ w).mean() < (X0_train @ w).mean():              # orient toward class 1
   w = -w

z_train = lda.transform(X_train).ravel() * np.sign(w @ lda.scalings_[:, 0])
z_test = lda.transform(X_test).ravel() * np.sign(w @ lda.scalings_[:, 0])
test_acc = float(lda.score(X_test, y_test) * 100)
print(f"LDA direction: [{w[0]:.3f}, {w[1]:.3f}]")
print(f"Test accuracy with 1 LDA feature: {test_acc:.1f}%")

threshold = round(float((z_train[y_train == 0].mean() + z_train[y_train == 1].mean()) / 2), 2) + 0.0
print(f"Threshold (midpoint of projected means): {threshold:.2f}")

anchor = (X0_train.mean(axis=0) + X1_train.mean(axis=0)) / 2
t_train = (X_train - anchor) @ w
t_test = (X_test - anchor) @ w
X_train_proj = anchor + np.outer(t_train, w)
X_test_proj = anchor + np.outer(t_test, w)

# ========================================
# 3. Plots: projected line + reduced features on it
# ========================================
plt.figure(figsize=(9, 5))

# 2D data, LDA line, every point -> its mapped point on the line
plt.scatter(X0_train[:, 0], X0_train[:, 1], c="steelblue", label="Train benign (0)", alpha=0.7)
plt.scatter(X1_train[:, 0], X1_train[:, 1], c="darkorange", label="Train malignant (1)", alpha=0.7)
plt.scatter(X0_test[:, 0], X0_test[:, 1], c="steelblue", marker="^", s=60,
         edgecolors="k", linewidths=0.5, label="Test benign (0)")
plt.scatter(X1_test[:, 0], X1_test[:, 1], c="darkorange", marker="^", s=60,
         edgecolors="k", linewidths=0.5, label="Test malignant (1)")
s = np.linspace(t_train.min() - 0.5, t_train.max() + 0.5, 100)
plt.plot(anchor[0] + s * w[0], anchor[1] + s * w[1], "r-", linewidth=2,
       label="LDA projected line")
for p, q in zip(X_train, X_train_proj):
   plt.plot([p[0], q[0]], [p[1], q[1]], "gray", linewidth=0.7, alpha=0.4)
for p, q in zip(X_test, X_test_proj):
   plt.plot([p[0], q[0]], [p[1], q[1]], "k-", linewidth=1.0, alpha=0.7)
plt.scatter(X_train_proj[:, 0], X_train_proj[:, 1], c="steelblue", s=20,
         marker="s", label="Train mapped on LDA", zorder=4)
plt.scatter(X_test_proj[:, 0], X_test_proj[:, 1], c="red", s=30,
         label="Test mapped on LDA", zorder=4)
plt.title(f"LDA: Points Projected onto Discriminant Line (test acc={test_acc:.1f}%)")
plt.xlabel("Tumor size (x1)")
plt.ylabel("Texture (x2)")
plt.legend(fontsize=8)
plt.grid(True, linestyle="--", alpha=0.6)

plt.tight_layout()
```

```
import os
out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "lda.png")
plt.savefig(out_path, dpi=150)
print(f"Plot saved to {out_path}")
plt.show()
```

Output

